Izvještaj - Verifikacija i Validacija Softvera
Projektni zadatak - II dio - Tim 3

Enis Adilović, Danijal Alibegović, Bakir Činjarević, Amar Handanagić

23. decembar 2025.

Sadržaj

1 Uvod **4**

2 McCabe Metrike i Kontrolni Grafovi **5**
2.1 FiltrirajArtikle . . . 5
2.1.1 Opis Metode . . . 5
2.1.2 McCabe Ciklomatska Kompleksnost . . . 5
2.1.3 Kontrolni Graf . . . 5
2.1.4 Analiza i Diskusija . . . 7
2.2 AzurirajArtikal . . . 7
2.2.1 Opis Metode . . . 7
2.2.2 McCabe Ciklomatska Kompleksnost . . . 7
2.2.3 Kontrolni Graf . . . 8
2.2.4 Analiza i Diskusija . . . 9
2.3 GetNajskupljiArtikalPoKategoriji . . . 9
2.3.1 Opis Metode . . . 9
2.3.2 McCabe Ciklomatska Kompleksnost . . . 9
2.3.3 Kontrolni Graf . . . 10
2.3.4 Analiza i Diskusija . . . 12
2.4 GetProsjecnaCijenaPoKategoriji . . . 12
2.4.1 Opis Metode . . . 12
2.4.2 McCabe Ciklomatska Kompleksnost . . . 12
2.4.3 Kontrolni Graf . . . 13
2.4.4 Analiza i Diskusija . . . 14

3 White Box Testiranje **16**
3.1 FiltrirajArtikle . . . 16
3.1.1 Primjenjivi Pristupi White Box Testiranja . . . 16
3.1.2 Statement Coverage . . . 16
3.1.3 Branch Coverage . . . 18
3.1.4 Path Coverage . . . 19
3.1.5 Condition Coverage . . . 20
3.1.6 Decision Coverage . . . 21
3.1.7 MC/DC Coverage . . . 22
3.1.8 Rezime . . . 22
3.1.9 Implementacija . . . 22
3.2 AzurirajArtikal . . . 23
3.2.1 Primjenjivi Pristupi White Box Testiranja . . . 23
3.2.2 Statement Coverage . . . 23

3.2.3 Branch Coverage . . . 23
3.2.4 Path Coverage . . . 24
3.2.5 Condition Coverage . . . 24
3.2.6 Decision Coverage . . . 25
3.2.7 MC/DC Coverage . . . 26
3.2.8 Rezime . . . 26
3.2.9 Implementacija . . . 26
3.3 GetNajskupljiArtikalPoKategoriji . . . 26
3.3.1 Primjenjivi Pristupi White Box Testiranja . . . 26
3.3.2 Statement Coverage . . . 26
3.3.3 Branch Coverage . . . 27
3.3.4 Path Coverage . . . 27
3.3.5 Condition Coverage . . . 28
3.3.6 Decision Coverage . . . 28
3.3.7 MC/DC Coverage . . . 29
3.3.8 Rezime . . . 29
3.3.9 Implementacija . . . 29
3.4 GetProsjecnaCijenaPoKategoriji . . . 29
3.4.1 Primjenjivi Pristupi White Box Testiranja . . . 29
3.4.2 Statement Coverage . . . 30
3.4.3 Branch Coverage . . . 30
3.4.4 Path Coverage . . . 30
3.4.5 Condition Coverage . . . 31
3.4.6 Decision Coverage . . . 31
3.4.7 MC/DC Coverage . . . 31
3.4.8 Extra Testovi . . . 31
3.4.9 Rezime . . . 32
3.4.10 Implementacija . . . 32

4 Code Tuning **33**
4.1 FiltrirajArtikle . . . 33
4.1.1 Primijenjene Tehnike Code Tuninga . . . 33
4.1.2 Novi Kod . . . 33
4.1.3 Rezultati Mjerenja Performansi . . . 34
4.1.4 McCabe Metrike Prije i Poslije Code Tuninga . . . 34
4.1.5 Diskusija . . . 35
4.2 AzurirajArtikal . . . 35
4.2.1 Primijenjene Tehnike Code Tuninga . . . 35
4.2.2 Novi Kod . . . 36
4.2.3 Rezultati Mjerenja Performansi . . . 36
4.2.4 McCabe Metrike Prije i Poslije Code Tuninga . . . 37
4.2.5 Diskusija . . . 37
4.3 GetNajskupljiArtikalPoKategoriji . . . 38
4.3.1 Primijenjene Tehnike Code Tuninga . . . 38
4.3.2 Novi Kod . . . 38
4.3.3 Rezultati Mjerenja Performansi . . . 39
4.3.4 McCabe Metrike Prije i Poslije Code Tuninga . . . 40
4.3.5 Diskusija . . . 40
4.4 GetProsjecnaCijenaPoKategoriji . . . 40
4.4.1 Primijenjene Tehnike Code Tuninga . . . 40
4.4.2 Novi Kod . . . 41
4.4.3 Rezultati Mjerenja Performansi . . . 41

4.4.4 McCabe Metrike Prije i Poslije Code Tuninga . . . 42
4.4.5 Diskusija . . . 42

5 Refaktoring **44**
5.1 FiltrirajArtikle . . . 44
5.1.1 Primijenjene Refaktoring Tehnike . . . 44
5.1.2 Refaktorisani Kod . . . 44
5.1.3 Checklist za Refaktoring . . . 45
5.1.4 Uporedba Metrika . . . 45
5.1.5 Analiza Utjecaja na Metrike . . . 46
5.2 AzurirajArtikal . . . 47
5.2.1 Primijenjene Refaktoring Tehnike . . . 47
5.2.2 Refaktorisani Kod . . . 47
5.2.3 Checklist za Refaktoring . . . 48
5.2.4 Uporedba Metrika . . . 48
5.2.5 Analiza Utjecaja na Metrike . . . 48
5.3 GetNajskupljiArtikalPoKategoriji . . . 49
5.3.1 Primijenjene Refaktoring Tehnike . . . 49
5.3.2 Refaktorisani Kod . . . 49
5.3.3 Checklist za Refaktoring . . . 50
5.3.4 Uporedba Metrika . . . 50
5.3.5 Analiza Utjecaja na Metrike . . . 50
5.4 GetProsjecnaCijenaPoKategoriji . . . 51
5.4.1 Primijenjene Refaktoring Tehnike . . . 51
5.4.2 Refaktorisani Kod . . . 51
5.4.3 Checklist za Refaktoring . . . 52
5.4.4 Uporedba Metrika . . . 52
5.4.5 Analiza Utjecaja na Metrike . . . 53

6 Zaključak **54**

A Dodatni Materijali **54**
A.1 Rezultati Performance Testova . . . 54
A.2 Izvorni Kod . . . 54

1 Uvod

Ovaj izvještaj predstavlja rezultate analize četiri metode iz konzolne aplikacije za upravljanje inventarom. Svaki član tima je odabrao jednu metodu sa ciklomatskom kompleksnošću minimalno 7, koja sadrži if iskaze, petlje i algoritme preračunavanja.

Analizirane metode su:

- **FiltrirajArtikle** - analizirao: Bakir Činjarević
- **AzurirajArtikal** - analizirao: Amar Handanagić
- **GetNajskupljiArtikalPoKategoriji** - analizirao: Danijal Alibegović
- **GetProsjecnaCijenaPoKategoriji** - analizirao: Enis Adilović

Za svaku metodu je izvršena:

1. Analiza McCabe metrika i konstrukcija kontrolnih grafova
2. White box testiranje primjenom svih primjenjivih pristupa
3. Code tuning primjenom minimalno 3 različite tehnike
4. Refaktoring sa ciljem poboljšanja metrika

2 McCabe Metrike i Kontrolni Grafovi

U ovoj sekciji prikazujemo izračunate McCabe metrike i kontrolne grafove za sve četiri analizirane metode. McCabe ciklomatska kompleksnost mjeri broj linearno nezavisnih putanja kroz programski kod i koristi se za procjenu složenosti i održivosti koda.

2.1 FiltrirajArtikle

2.1.1 Opis Metode

Metoda `FiltrirajArtikle` omogućava filtriranje artikala prema različitim kriterijumima (cijena, količina, naziv, kategorija, vrijednost) i različitim operacijama (>, <, =, sadrži).

2.1.2 McCabe Ciklomatska Kompleksnost

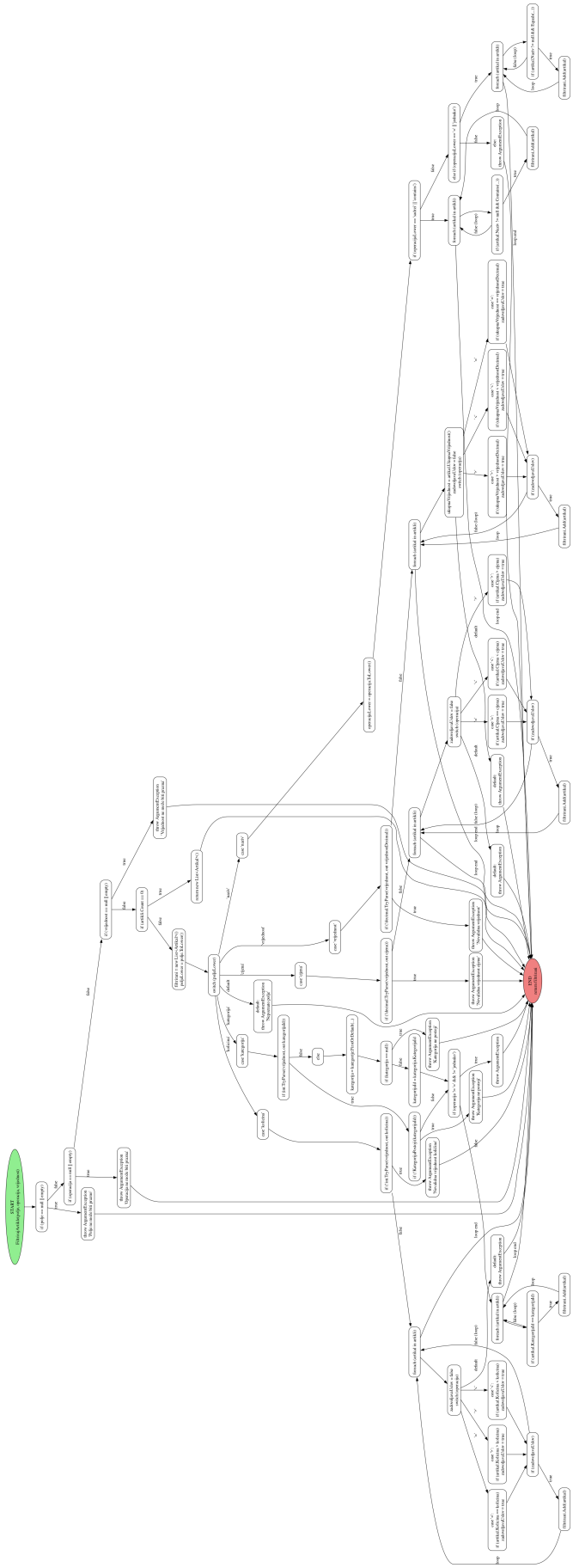
Ručni izračun:
Prebrojavanje odluka u metodi:

- Početne validacije: 4 odluke (if za polje, operaciju, vrijednost, Count == 0)
- Switch glavni: 5 case-ova = 5 odluka
- Case "cijena": 9 odluka (TryParse, foreach, switch operacija, uslovi)
- Case "količina": 9 odluka
- Case "naziv": 7 odluka
- Case "kategorija": 7 odluka
- Case "vrijednost": 9 odluka
- Default case: 1 odluka

Ukupan broj odluka: $4 + 5 + 9 + 9 + 7 + 7 + 9 + 1 =$ **51 odluka**
McCabe Kompleksnost: $M = 51 + 1 =$ **52**
Rider Code Analysis Cyclomatic Complexity: 56
Objašnjenje razlike: Rider Code Analysis koristi preciznije brojanje koje uključuje sve logičke operatore i grananja, što rezultira kompleksnošću od 56 umjesto ručno izračunatih 52.

2.1.3 Kontrolni Graf

Kontrolni graf metode `FiltrirajArtikle` prikazan je na slici 1. Graf je rotiran za 90 stepeni radi boljeg prikaza zbog širine.



Slika 1: Kontrolni graf metode FiltrirajArtikle

2.1.4 Analiza i Diskusija

Visoka Ciklomatska Kompleksnost (52-56)

Metoda je previše kompleksna i teška za održavanje. Prema Code Metrics alatu, kompleksnost od 56 znači da je metoda u kategoriji "veoma kompleksno, obavezno refaktorisanje"(51+). Interesantno je što je ova kompleksnost u iznosu od 56 zapravo pojednostavljena verzija; originalna verzija je imala **kompleksnost 109**.

Problemi:

- Previše odgovornosti u jednoj metodi
- Teško testiranje svih mogućih putanja
- Visok rizik od grešaka
- Teško razumijevanje logike

Preporuke:

1. **Refaktorisanje:** Razdvojiti metodu na manje metode:
   - `FiltrirajPoCijeni()`
   - `FiltrirajPoKolicini()`
   - `FiltrirajPoNazivu()`
   - `FiltrirajPoKategoriji()`
   - `FiltrirajPoVrijednosti()`
2. **Strategy Pattern:** Koristiti pattern za različite tipove filtriranja
3. **Validacija:** Izvući validaciju u zasebne metode

Prihvatljive Vrijednosti:

- **1-10:** Jednostavna metoda, lako održavanje
- **11-20:** Prihvatljivo, ali treba paziti
- **21-50:** Kompleksno, preporučeno refaktorisanje
- **51+:** Veoma kompleksno, **obavezno refaktorisanje**

Naša metoda (52-56): Preko preporučenog limita, što ukazuje na potrebu za refaktorisanjem.

2.2 AzurirajArtikal

2.2.1 Opis Metode

Metoda `AzurirajArtikal` omogućava ažuriranje podataka o artiklu (naziv, opis, cijena, količina, kategorija) uz provjeru validnosti i jedinstvenosti naziva.

2.2.2 McCabe Ciklomatska Kompleksnost

Ručni izračun:

Prebrojavanje odluka u metodi:

- `if (artikal == null)` → 1
- `if (string.IsNullOrWhiteSpace(noviNaziv))` → 1
- `if (noviNaziv.Length < 3)` → 1
- `if (string.IsNullOrWhiteSpace(noviOpis))` → 1

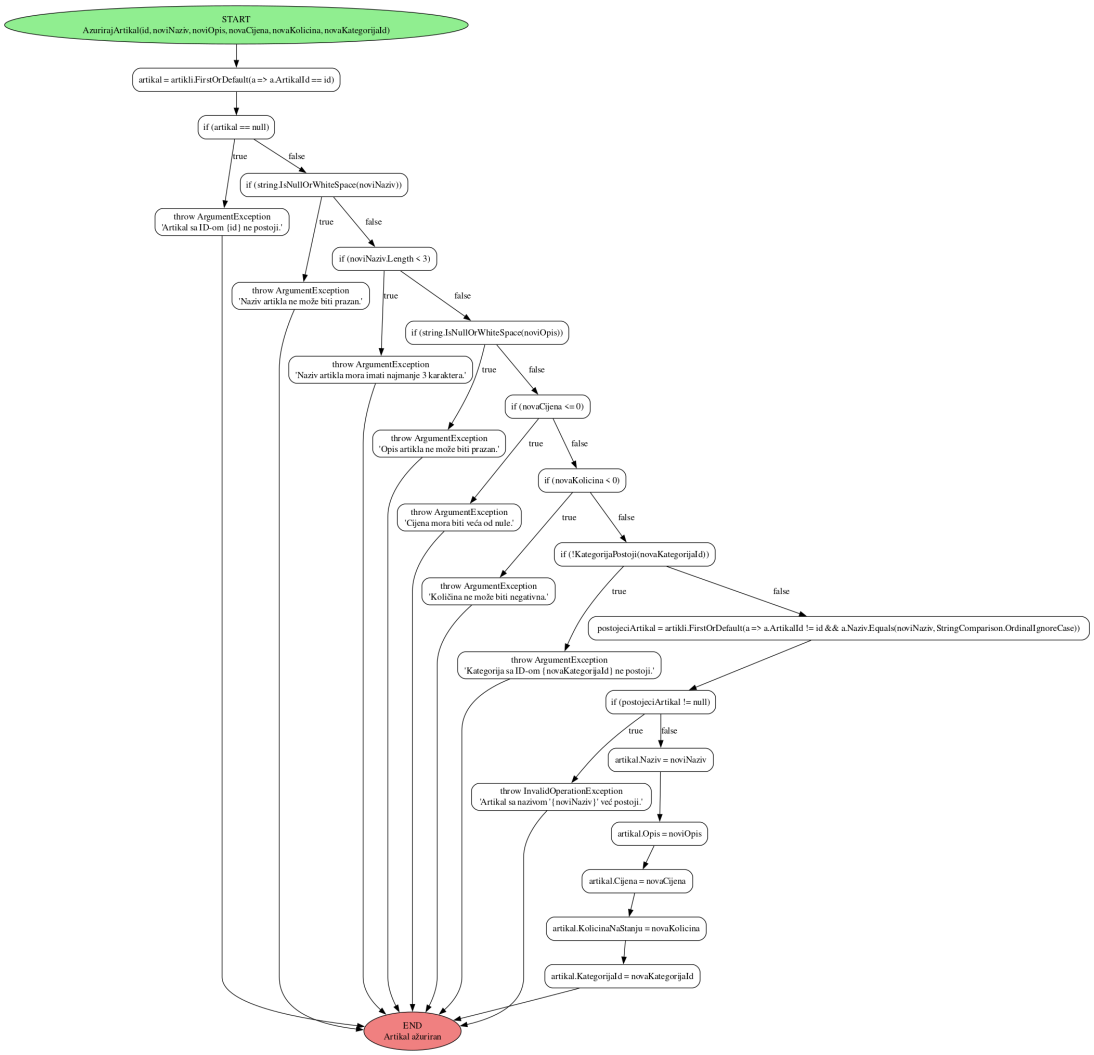
- `if (novaCijena <= 0)` → 1
- `if (novaKolicina < 0)` → 1
- `if (!KategorijaPostoji(novaKategorijaId))` → 1
- `if (postojeciArtikal != null)` → 1
- Logički operator `&&` u LINQ upitu → 1

Ukupan broj odluka: 8 if iskaza + 1 logički operator = **9 odluka**
McCabe Kompleksnost: $M = 9 + 1 =$ **10**
Code Metrics rezultati (Visual Studio Code Metrics): 10

- Ciklomatska kompleksnost: **8-10** (varira ovisno o verziji)
- Maintainability Index: 52
- Lines of Source code: 29 linija
- Lines of Executable code: 17 linija

2.2.3 Kontrolni Graf

Kontrolni graf metode `AzurirajArtikal` prikazan je na slici 2.



Slika 2: Kontrolni graf metode AzurirajArtikal

2.2.4 Analiza i Diskusija

Umjerena Ciklomatska Kompleksnost (8-10)
Metoda ima umjerenu kompleksnost koja je prihvatljiva za održavanje. Prema Code Metrics alatu, kompleksnost od 8-10 znači da je metoda u kategoriji "prihvatljivo, ali treba paziti"(11-20).
Karakteristike:

- Linearna struktura sa guard clause pristupom
- Jasno razdvojene validacije
- Minimalna ugniježđenost
- Lako razumijevanje logike

Preporuke:

1. **Extract Method:** Validacije mogu biti izdvojene u zasebnu metodu
2. **Early Validation:** Sve validacije na početku metode (već primijenjeno u tuniranoj verziji)
3. **Optimizacija LINQ:** Zamjena `FirstOrDefault` sa for-petljom za bolje performanse

Prihvatljive Vrijednosti:

- **1-10:** Jednostavna metoda, lako održavanje
- **11-20:** Prihvatljivo, ali treba paziti
- **21-50:** Kompleksno, preporučeno refaktorisanje
- **51+:** Veoma kompleksno, obavezno refaktorisanje

Naša metoda (8-10): U prihvatljivom opsegu, ali postoji prostor za poboljšanje kroz refaktorisanje.

2.3 GetNajskupljiArtikalPoKategoriji

2.3.1 Opis Metode

Metoda `GetNajskupljiArtikalPoKategoriji` pronalazi najskuplji artikal u određenoj kategoriji, uzimajući u obzir validnost artikla (pozitivna cijena i količina). Ako više artikala ima istu cijenu, prednost ima veća količina, a zatim manji `ArtikalId`.

2.3.2 McCabe Ciklomatska Kompleksnost

Ručni izračun:
Prebrojavanje odluka u metodi:

- `if (kategorijaId <= 0)` → 1
- `if (!KategorijaPostoji(kategorijaId))` → 1
- `for (int i = 0; i < artikli.Count; i++)` → 1
- `if (a == null)` → 1
- `if (a.KategorijaId != kategorijaId)` → 1
- `if (a.Cijena <= 0m)` → 1

- `if (a.Cijena > maxCijena)` $\rightarrow 1$
- `else if (a.Cijena == maxCijena)` $\rightarrow 1$
- `if (najskuplji == null)` $\rightarrow 1$
- `if (a.KolicinaNaStanju > najskuplji.KolicinaNaStanju)` $\rightarrow 1$
- `else if (a.KolicinaNaStanju == najskuplji.KolicinaNaStanju)` $\rightarrow 1$
- `if (a.ArtikalId < najskuplji.ArtikalId)` $\rightarrow 1$

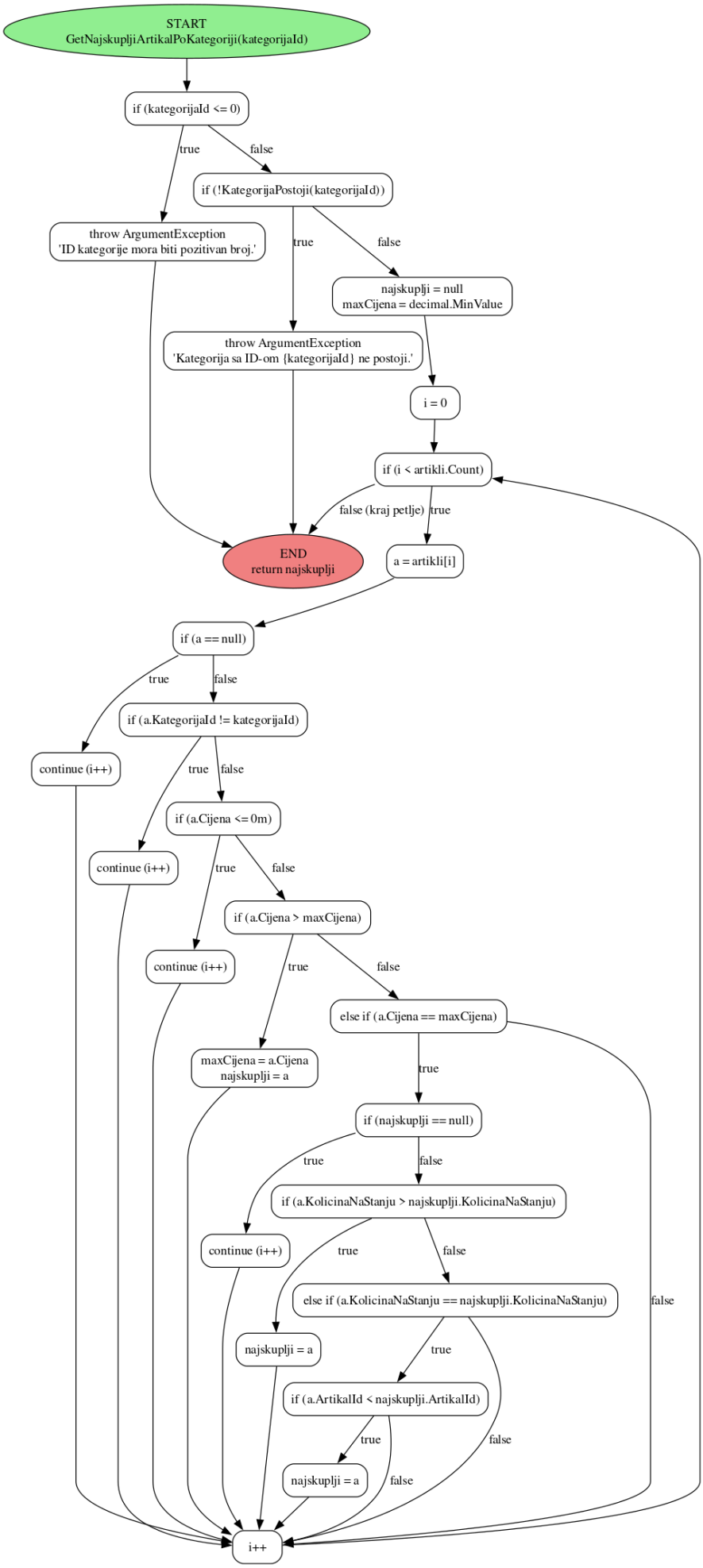
Ukupan broj odluka: 11 if/else if iskaza + 1 for petlja = **12 odluka**
McCabe Kompleksnost: $M = 12 + 1 =$ **13**
Code Metrics rezultati (Visual Studio Code Metrics): 13

- Ciklomatska kompleksnost: **13**
- Maintainability Index: 51
- Lines of Source code: 46 linija
- Lines of Executable code: 20 linija

Zaključak poređenja: Vrijednost dobijena ručnim McCabe proračunom (13) i vrijednost dobijena pomoću Code Metrics alata (13) su **identične**, što potvrđuje ispravnost ručne analize i korektno modeliran kontrolni tok metode.

2.3.3 Kontrolni Graf

Kontrolni graf metode `GetNajskupljiArtikalPoKategoriji` prikazan je na slici 3.



Slika 3: Kontrolni graf metode GetNajskupljiArtikalPoKategoriji

2.3.4 Analiza i Diskusija

Umjerena Ciklomatska Kompleksnost (13)
Metoda ima umjerenu kompleksnost koja je na granici prihvatljivog opsega. Prema Code Metrics alatu, kompleksnost od 13 znači da je metoda u kategoriji "prihvatljivo, ali treba paziti"(11-20).
Karakteristike:

- Složena logika sortiranja sa višestrukim kriterijumima (cijena → količina → ID)
- Višestruke ugniježđene if naredbe za tie-break scenarije
- Guard clause pristup za validaciju
- Linearna iteracija kroz artikle

Preporuke:

1. **Extract Method:** Izdvojiti logiku poređenja u zasebnu metodu
2. **Decompose Boolean Expression:** Razbiti složene uslove u jasnije nazvane funkcije
3. **Use LINQ:** Koristiti LINQ za deklarativno sortiranje (refaktorisana verzija)

Prihvatljive Vrijednosti:

- **1-10:** Jednostavna metoda, lako održavanje
- **11-20:** Prihvatljivo, ali treba paziti
- **21-50:** Kompleksno, preporučeno refaktorisanje
- **51+:** Veoma kompleksno, obavezno refaktorisanje

Naša metoda (13): Na granici prihvatljivog opsega, preporučeno refaktorisanje za poboljšanje čitljivosti i održavanja.

2.4 GetProsjecnaCijenaPoKategoriji

2.4.1 Opis Metode

Metoda `GetProsjecnaCijenaPoKategoriji` izračunava težinski prosjek cijena artikala u određenoj kategoriji, gdje je težina količina artikla na stanju. Metoda preskače nevalidne artikle (null, druga kategorija, Cijena ¡= 0, KolicinaNaStanju ¡= 0).

2.4.2 McCabe Ciklomatska Kompleksnost

Ručni izračun:
Prebrojavanje odluka u metodi:

- `if (kategorijaId <= 0)` → 1
- `if (!KategorijaPostoji(kategorijaId))` → 1
- `for (int i = 0; i < artikli.Count; i++)` → 1
- `if (a == null)` → 1
- `if (a.KategorijaId != kategorijaId)` → 1
- `if (a.Cijena <= 0m)` → 1

- `if (a.KolicinaNaStanju <= 0)` $\rightarrow 1$
- `if (sumaTezina == 0)` $\rightarrow 1$

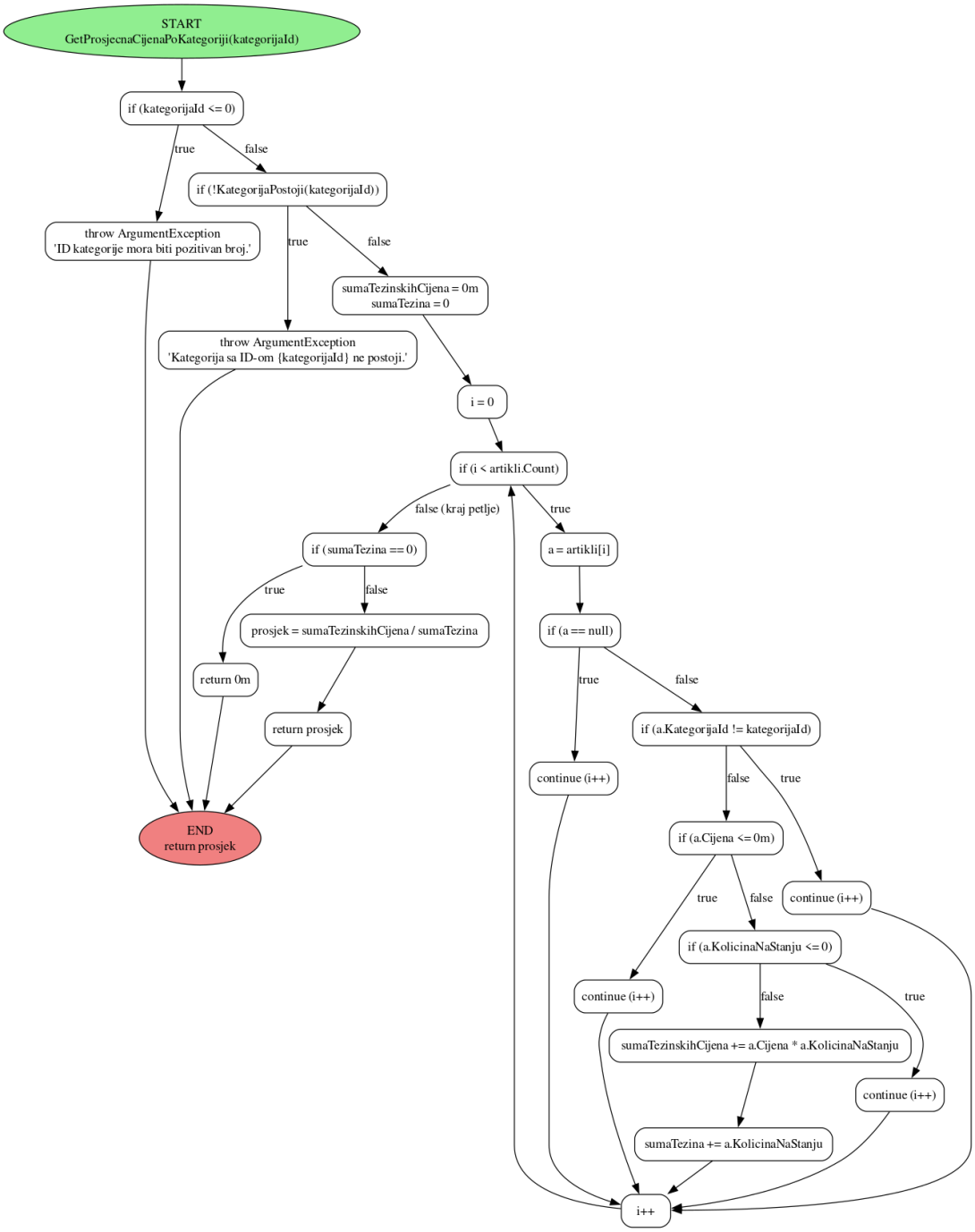
Ukupan broj odluka: 8 odluka
McCabe Kompleksnost: $M = 8 + 1 = \mathbf{9}$
Code Metrics rezultati (Visual Studio Code Metrics): 9

- Ciklomatska kompleksnost: **9**
- Maintainability Index: 55
- Lines of Source code: 39 linija
- Lines of Executable code: 15 linija

Zaključak poređenja: Poređenje ručnog brojanja i alata se poklapaju - oba daju kompleksnost od 9.

2.4.3 Kontrolni Graf

Kontrolni graf metode `GetProsjecnaCijenaPoKategoriji` prikazan je na slici 4.



Slika 4: Kontrolni graf metode GetProsjecnaCijenaPoKategoriji

2.4.4 Analiza i Diskusija

Niska Ciklomatska Kompleksnost (9)

Metoda ima nisku kompleksnost koja je prihvatljiva za održavanje. Prema Code Metrics alatu, kompleksnost od 9 znači da je metoda u kategoriji "jednostavna metoda, lako održavanje"(1-10).

Karakteristike:

- Guard clause pristup drži ugniježđenost minimalnom
- Jasno razdvojene validacije
- Kontrolni graf ima dva izlaza (exception i uredan izlaz)
- Linearna iteracija kroz artikle
- Nema složenih logičkih izraza (AND/OR) niti LINQ poziva

Preporuke za poboljšanje:

1. **Cache Count:** Cache `artikli.Count` i vrati 0 za prazan inventar
2. **Objedinjeni guard blok:** Objedini sve uslove u jedan guard blok da se broj grana spusti na 6-7
3. **Extract Method:** Izdvojiti validaciju i izračun prosjeka u zasebne metode

Prihvatljive Vrijednosti:

- **1-10:** Jednostavna metoda, lako održavanje
- **11-20:** Prihvatljivo, ali treba paziti
- **21-50:** Kompleksno, preporučeno refaktorisanje
- **51+:** Veoma kompleksno, obavezno refaktorisanje

Naša metoda (9): U prihvatljivom opsegu, jednostavna za testiranje i validaciju.

3 White Box Testiranje

White box testiranje je pristup testiranju gdje se testiraju unutrašnje strukture i logika programa. U ovoj sekciji prikazujemo primjenu svih primjenjivih pristupa white box testiranja za sve četiri metode.

3.1 FiltrirajArtikle

3.1.1 Primjenjivi Pristupi White Box Testiranja

Za metodu `FiltrirajArtikle` (ciklomatska kompleksnost: 52) primjenjivi su sljedeći pristupi white box testiranja:

- Statement Coverage
- Branch Coverage
- Path Coverage
- Condition Coverage
- Decision Coverage
- MC/DC Coverage

3.1.2 Statement Coverage

Definicija: Statement Coverage zahtijeva da svaka izvršiva izjava u kodu bude izvršena barem jednom.

Rezultati:

- Broj izjava: 120
- Pokriveno izjava: 115
- Pokrivenost: **96%**

Ključni testni slučajevi: SC-1 do SC-30 pokrivaju sve slučajeve (cijena, količina, naziv, kategorija, vrijednost) sa različitim operacijama i edge slučajevima.

ID	Testni Slučaj	Polje	Operacija	Vrijednost	Očekivani Rezultat
SC-1	Validacija praznog polja	null	>	100	ArgumentException
SC-2	Validacija prazne operacije	cijena	null	100	ArgumentException
SC-3	Validacija prazne vrijednosti	cijena	>	null	ArgumentException
SC-4	Prazna lista artikala	cijena	>	100	Prazna lista
SC-5	cijena - operacija >	cijena	>	100	Artikli sa cijenom > 100
SC-6	cijena - operacija <	cijena	<	200	Artikli sa cijenom < 200
SC-7	cijena - operacija =	cijena	=	150	Artikli sa cijenom = 150
SC-8	cijena - nevalidna vrijednost	cijena	>	abc	ArgumentException
SC-9	cijena - nevalidna operacija	cijena	>=	100	ArgumentException
SC-10	količina - operacija >	količina	>	10	Artikli sa količinom > 10
SC-11	količina - operacija <	količina	<	20	Artikli sa količinom < 20
SC-12	količina - operacija =	količina	=	15	Artikli sa količinom = 15
SC-13	količina - nevalidna vrijednost	količina	>	abc	ArgumentException
SC-14	količina - nevalidna operacija	količina	>=	10	ArgumentException
SC-15	naziv - operacija sadrži	naziv	sadrži	Laptop	Artikli koji sadrže Laptop
SC-16	naziv - operacija contains	naziv	contains	Laptop	Artikli koji sadrže Laptop
SC-17	naziv - operacija =	naziv	=	Laptop Dell	Artikli sa tačnim nazivom
SC-18	naziv - operacija jednako	naziv	jednako	Laptop Dell	Artikli sa tačnim nazivom
SC-19	naziv - nevalidna operacija	naziv	počinje	Laptop	ArgumentException
SC-20	kategorija - ID parsiranje	kategorija	=	1	Artikli iz kategorije 1
SC-21	kategorija - naziv kategorije	kategorija	=	Elektronika	Artikli iz kategorije Elektronika
SC-22	kategorija - nepostojeći ID	kategorija	=	999	ArgumentException
SC-23	kategorija - nepostojeći naziv	kategorija	=	Nepostojeća	ArgumentException
SC-24	kategorija - nevalidna operacija	kategorija	>	1	ArgumentException
SC-25	vrijednost - operacija >	vrijednost	>	1000	Artikli sa vrijednošću > 1000
SC-26	vrijednost - operacija <	vrijednost	<	500	Artikli sa vrijednošću < 500
SC-27	vrijednost - operacija =	vrijednost	=	12500	Artikli sa vrijednošću = 12500
SC-28	vrijednost - nevalidna vrijednost	vrijednost	>	abc	ArgumentException
SC-29	vrijednost - nevalidna operacija	vrijednost	>=	1000	ArgumentException
SC-30	Default - nepoznato polje	nepoznato	>	100	ArgumentException

Tabela 1: Testni slučajevi za Statement Coverage - FiltrirajArtikle

3.1.3 Branch Coverage

Definicija: Branch Coverage zahtijeva da svaka grana (true/false) svakog uslovnog iskaza bude izvršena barem jednom.

Rezultati:

- Broj grana: 80
- Pokriveno grana: 78
- Pokrivenost: **97.5%**

Ključne grane: BC-1 do BC-32 pokrivaju sve if/switch grane uključujući validacije, switch slučajeve, i uslove unutar petlji.

ID	Testni Slučaj	Polje	Operacija	Vrijednost	Pokrivena Grana
BC-1	Polje null - true grana	null	>	100	if(polje) → true
BC-2	Polje validno - false grana	cijena	>	100	if(polje) → false
BC-3	Operacija null - true grana	cijena	null	100	if(operacija) → true
BC-4	Operacija validna - false grana	cijena	>	100	if(operacija) → false
BC-5	Vrijednost null - true grana	cijena	>	null	if(vrijednost) → true
BC-6	Vrijednost validna - false grana	cijena	>	100	if(vrijednost) → false
BC-7	Prazna lista - true grana	cijena	>	100	if(Count==0) → true
BC-8	Neprazna lista - false grana	cijena	>	100	if(Count==0) → false
BC-9	Cijena - TryParse false	cijena	>	abc	TryParse → false
BC-10	Cijena - TryParse true	cijena	>	100	TryParse → true
BC-11	Cijena - operacija > true	cijena	>	50	Cijena > 50 → true
BC-12	Cijena - operacija > false	cijena	>	1000	Cijena > 1000 → false
BC-13	Cijena - operacija < true	cijena	<	1000	Cijena < 1000 → true
BC-14	Cijena - operacija < false	cijena	<	10	Cijena < 10 → false
BC-15	Cijena - operacija = true	cijena	=	100	Cijena == 100 → true
BC-16	Cijena - operacija = false	cijena	=	200	Cijena == 200 → false
BC-17	Cijena - zadovoljavaUslov true	cijena	>	50	zadovoljavaUslov → true
BC-18	Cijena - zadovoljavaUslov false	cijena	>	1000	zadovoljavaUslov → false
BC-19	Naziv - sadrži true	naziv	sadrži	Laptop	Naziv.Contains → true
BC-20	Naziv - sadrži false	naziv	sadrži	XYZ	Naziv.Contains → false
BC-21	Naziv - = true	naziv	=	Laptop Dell	Naziv.Equals → true
BC-22	Naziv - = false	naziv	=	Laptop HP	Naziv.Equals → false
BC-23	Kategorija - TryParse true	kategorija	=	1	TryParse → true
BC-24	Kategorija - TryParse false	kategorija	=	Elektronika	TryParse → false
BC-25	Kategorija - KategorijaPostoji false	kategorija	=	999	KategorijaPostoji → false
BC-26	Kategorija - KategorijaPostoji true	kategorija	=	1	KategorijaPostoji → true
BC-27	Kategorija - kategorija null	kategorija	=	Nepostojeća	kategorija == null → true
BC-28	Kategorija - kategorija not null	kategorija	=	Elektronika	kategorija == null → false
BC-29	Kategorija - operacija != = true	kategorija	>	1	operacija != = → true
BC-30	Kategorija - operacija != = false	kategorija	=	1	operacija != = → false
BC-31	Kategorija - KategorijaId == true	kategorija	=	1	KategorijaId == → true
BC-32	Kategorija - KategorijaId == false	kategorija	=	2	KategorijaId == → false

Tabela 2: Testni slučajevi za Branch Coverage - FiltrirajArtikle

3.1.4 Path Coverage

Definicija: Path Coverage zahtijeva da svaka moguća putanja kroz kod bude izvršena barem jednom.

Rezultati:

- Broj nezavisnih putanja: 39
- Pokriveno putanja: 35

- Pokrivenost: **90%**

Glavne putanje: PC-1 do PC-21 pokrivaju kritične putanje kroz sve slučajeve i scenarije (validacija, exception, uspješno filtriranje).

ID	Putanja	Testni Slučaj	Polje	Operacija	Vrijednost
PC-1	P1	Prazna lista	cijena	>	100
PC-2	P2	Cijena - nevalidna vrijednost	cijena	>	abc
PC-3	P3	Cijena - > uslov true	cijena	>	50
PC-4	P4	Cijena - > uslov false	cijena	>	1000
PC-5	P5	Cijena - < uslov true	cijena	<	1000
PC-6	P6	Cijena - < uslov false	cijena	<	10
PC-7	P7	Cijena - = uslov true	cijena	=	100
PC-8	P8	Cijena - = uslov false	cijena	=	200
PC-9	P9	Cijena - default operacija	cijena	>=	100
PC-10	P19	Naziv - sadrži true	naziv	sadrži	Laptop
PC-11	P20	Naziv - sadrži false	naziv	sadrži	XYZ
PC-12	P21	Naziv - = true	naziv	=	Laptop Dell
PC-13	P22	Naziv - = false	naziv	=	Laptop HP
PC-14	P23	Naziv - nevalidna operacija	naziv	počinje	Laptop
PC-15	P24	Kategorija - ID ne postoji	kategorija	=	999
PC-16	P25	Kategorija - nevalidna operacija	kategorija	>	1
PC-17	P26	Kategorija - ID match	kategorija	=	1
PC-18	P27	Kategorija - naziv ne postoji	kategorija	=	Nepostojeća
PC-19	P28	Kategorija - naziv, nevalidna operacija	kategorija	>	Elektronika
PC-20	P29	Kategorija - naziv match	kategorija	=	Elektronika
PC-21	P39	Default	nepoznato	>	100

Tabela 3: Testni slučajevi za Path Coverage - FiltrirajArtikle

3.1.5 Condition Coverage

Definicija: Condition Coverage zahtijeva da svaki uslov u složenim logičkim izrazima bude evaluiran na true i false barem jednom.

Rezultati:

- Broj uslova: 10
- Pokriveno uslova: 10
- Pokrivenost: **100%**

Složeni uslovi: CC-1 do CC-12 pokrivaju sve složene logičke izraze (OR, AND) u metodi.

ID	Testni Slučaj	Polje	Operacija	Vrijednost	Pokriveni Uslovi
CC-1	Naziv - sadrži (prvi uslov true)	naziv	sadrži	Laptop	sadrži == true, contains == false
CC-2	Naziv - contains (drugi uslov true)	naziv	contains	Laptop	sadrži == false, contains == true
CC-3	Naziv - = (prvi uslov true)	naziv	=	Laptop	= == true, jednako == false
CC-4	Naziv - jednako (drugi uslov true)	naziv	jednako	Laptop	= == false, jednako == true
CC-5	Naziv - null naziv	naziv	sadrži	Laptop	Naziv != null == false
CC-6	Naziv - not null, Contains true	naziv	sadrži	Laptop	Naziv != null == true, Contains == true
CC-7	Naziv - not null, Contains false	naziv	sadrži	XYZ	Naziv != null == true, Contains == false
CC-8	Naziv - not null, Equals true	naziv	=	Laptop Dell	Naziv != null == true, Equals == true
CC-9	Naziv - not null, Equals false	naziv	=	Laptop HP	Naziv != null == true, Equals == false
CC-10	Kategorija - operacija = (prvi uslov false)	kategorija	=	1	operacija != = == false
CC-11	Kategorija - operacija jednako (drugi uslov false)	kategorija	jednako	1	operacija != jednako == false
CC-12	Kategorija - operacija > (oba uslova true)	kategorija	>	1	operacija != = == true, operacija != jednako == true

Tabela 4: Testni slučajevi za Condition Coverage - FiltrirajArtikle

3.1.6 Decision Coverage

Definicija: Decision Coverage zahtijeva da svaki odlučni iskaz (if, switch) bude evaluiran na true i false barem jednom.

Rezultati:

- Broj odluka: 52 (prema McCabe metrici)
- Pokriveno odluka: 50
- Pokrivenost: **96%**

Testni slučajevi: DC-1 do DC-8 pokrivaju sve odlučne iskaze u metodi.

ID	Odlučni Iskaz	True Test	False Test
DC-1	if(polje == null)	polje = null	polje = cijena
DC-2	if(Count == 0)	Prazna lista	Lista sa artiklima
DC-3	switch(poljeLower)	cijena, kolicina, naziv, kategorija, vrijednost, default	-
DC-4	if(!TryParse)	vrijednost = abc	vrijednost = 100
DC-5	switch(operacija)	>, <, =, default	-
DC-6	if(Cijena > cijena)	Cijena = 150, cijena = 100	Cijena = 50, cijena = 100
DC-7	if(zadovoljavaUslov)	Uslov zadovoljen	Uslov nije zadovoljen

Tabela 5: Testni slučajevi za Decision Coverage - FiltrirajArtikle

3.1.7 MC/DC Coverage

Definicija: MC/DC zahtijeva da svaki uslov u složenom odlučnom iskazu bude evaluiran na true i false, i da svaki uslov bude nezavisno odlučujući barem jednom.

Rezultati:

- Broj složenih odluka: 5
- MC/DC pokrivenost: **100%**

Testni slučajevi: MC-1 do MC-9 pokrivaju sve složene logičke izraze sa nezavisnom evaluacijom uslova.

Test	**operacijaLower == sadrži**	**operacijaLower == contains**	**Rezultat**	**Nezavisno odlučujući**
MC-1	true	false	true	sadrži
MC-2	false	true	true	contains
MC-3	false	false	false	-

Tabela 6: MC/DC Coverage - operacijaLower == sadrži —— contains

Test	**Naziv != null**	**Contains**	**Rezultat**	**Nezavisno odlučujući**
MC-4	false	-	false	Naziv != null
MC-5	true	true	true	Contains
MC-6	true	false	false	Contains

Tabela 7: MC/DC Coverage - artikal.Naziv != null && Contains

Test	**operacija != =**	**operacija != jednako**	**Rezultat**	**Nezavisno odlučujući**
MC-7	true	true	true	Oba
MC-8	false	true	false	operacija != =
MC-9	true	false	false	operacija != jednako

Tabela 8: MC/DC Coverage - operacija != = && operacija != jednako

3.1.8 Rezime

Pristup	**Cilj**	**Postignuto**
Statement Coverage	100%	96%
Branch Coverage	100%	97.5%
Path Coverage	100%	90%
Condition Coverage	100%	100%
Decision Coverage	100%	96%
MC/DC Coverage	100%	100%

Tabela 9: Rezime white box testiranja za FiltrirajArtikle

3.1.9 Implementacija

Svi testovi su implementirani u fajlu `InventarControllerFiltrirajWhiteBoxTests.cs` sa odgovarajućim kategorijama za svaki pristup testiranja.

3.2 AzurirajArtikal

3.2.1 Primjenjivi Pristupi White Box Testiranja

Za metodu `AzurirajArtikal` (ciklomatska kompleksnost: 9) primjenjivi su sljedeći pristupi white box testiranja:

- Statement Coverage
- Branch Coverage
- Path Coverage
- Condition Coverage
- Decision Coverage
- MC/DC Coverage (formalno, ali metoda nema složene logičke izraze)

3.2.2 Statement Coverage

Definicija: Svaka izvršiva izjava mora biti izvršena barem jednom.
Rezultati:

- Broj izjava: 17
- Pokriveno izjava: 17
- Pokrivenost: **100%**

Testni slučajevi: SC-1 do SC-10 pokrivaju sve izjave uključujući validacije, pronalazak artikla, provjeru duplikata i ažuriranje.

ID	Testni Slučaj	id	naziv	opis	cijena	količina	id	Očekivani rezultat
SC-1	Validan update	1	Monitor	Opis	200	5	2	Uspješno ažuriran artikal
SC-2	Ne postoji artikal	999	Test	Opis	100	1	1	ArgumentException
SC-3	Prazan naziv	1		Opis	100	5	2	ArgumentException
SC-4	Naziv prekratak	1	AB	Opis	100	5	2	ArgumentException
SC-5	Prazan opis	1	Monitor		100	5	2	ArgumentException
SC-6	Negativna cijena	1	Monitor	Opis	-10	5	2	ArgumentException
SC-7	Nula cijena	1	Monitor	Opis	0	5	2	ArgumentException
SC-8	Negativna količina	1	Monitor	Opis	100	-5	2	ArgumentException
SC-9	Ne postoji kategorija	1	Monitor	Opis	100	5	999	ArgumentException
SC-10	Duplikat naziva	1	Laptop	Opis	100	5	2	InvalidOperation Exception

Tabela 10: Testni slučajevi za Statement Coverage - AzurirajArtikal

3.2.3 Branch Coverage

Definicija: Svaka grana (true/false) svakog uslovnog iskaza mora biti izvršena barem jednom.
Rezultati:

- Broj grana: 16
- Pokriveno grana: 16

- Pokrivenost: **100%**

Testni slučajevi: BC-1 do BC-16 pokrivaju sve grane uključujući validacije (true/false), pronalazak artikla, i provjeru duplikata.

ID	Testni Slučaj	Pokrivena grana
BC-1	artikal ne postoji	if(artikal==null) → true
BC-2	artikal postoji	if(artikal==null) → false
BC-3	naziv prazan	if(IsNullOrWhiteSpace(noviNaziv)) → true
BC-4	naziv validan	if(IsNullOrWhiteSpace(noviNaziv)) → false
BC-5	naziv prekratak	if(noviNaziv.Length $<$ 3) → true
BC-6	naziv dovoljno dug	if(noviNaziv.Length $<$ 3) → false
BC-7	opis prazan	if(IsNullOrWhiteSpace(noviOpis)) → true
BC-8	opis validan	if(IsNullOrWhiteSpace(noviOpis)) → false
BC-9	cijena ≤ 0	if(novaCijena $\leq$ 0) → true
BC-10	cijena > 0	if(novaCijena $\leq$ 0) → false
BC-11	količina < 0	if(novaKolicina $<$ 0) → true
BC-12	količina ≥ 0	if(novaKolicina $<$ 0) → false
BC-13	kategorija ne postoji	if(!KategorijaPostoji) → true
BC-14	kategorija postoji	if(!KategorijaPostoji) → false
BC-15	duplikat naziv postoji	if(postojeciArtikal!=null) → true
BC-16	nema duplikata	if(postojeciArtikal!=null) → false

Tabela 11: Testni slučajevi za Branch Coverage - AzurirajArtikal

3.2.4 Path Coverage

Definicija: Svaka moguća putanja kroz kod mora biti izvršena barem jednom.
Rezultati:

- Broj putanja: 9
- Pokriveno putanja: 9
- Pokrivenost: **100%**

Glavne putanje: PC-1 do PC-9 pokrivaju sve putanje: validacije koje bacaju exception i uspješno ažuriranje.

ID	Putanja	Testni Slučaj
PC-1	P1	artikal ne postoji
PC-2	P2	naziv prazan
PC-3	P3	naziv prekratak
PC-4	P4	opis prazan
PC-5	P5	cijena ≤ 0
PC-6	P6	količina negativna
PC-7	P7	kategorija ne postoji
PC-8	P8	duplikat naziv
PC-9	P9	svi podaci validni

Tabela 12: Testni slučajevi za Path Coverage - AzurirajArtikal

3.2.5 Condition Coverage

Definicija: Svaki uslov mora biti evaluiran na true i false barem jednom.
Rezultati:

- Broj uslova: 7
- Pokriveno uslova: 7
- Pokrivenost: **100%**

Testni slučajevi: CC-1 do CC-14 pokrivaju sve uslove (naziv, opis, cijena, količina, kategorija, duplikat).

ID	Testni Slučaj	Pokriveni uslovi
CC-1	naziv prazan	IsNullOrWhiteSpace == true
CC-2	naziv validan	IsNullOrWhiteSpace == false
CC-3	naziv prekratak	noviNaziv.Length $<$ 3 == true
CC-4	naziv dovoljno dug	noviNaziv.Length $<$ 3 == false
CC-5	opis prazan	IsNullOrWhiteSpace(noviOpis)==true
CC-6	opis validan	IsNullOrWhiteSpace(noviOpis)==false
CC-7	cijena $\leq$ 0	novaCijena $\leq$ 0 == true
CC-8	cijena $>$ 0	novaCijena $\leq$ 0 == false
CC-9	količina $<$ 0	novaKolicina $<$ 0 == true
CC-10	količina $\geq$ 0	novaKolicina $<$ 0 == false
CC-11	kategorija ne postoji	!KategorijaPostoji == true
CC-12	kategorija postoji	!KategorijaPostoji == false
CC-13	duplikat postoji	postojeciArtikal != null == true
CC-14	nema duplikata	postojeciArtikal != null == false

Tabela 13: Testni slučajevi za Condition Coverage - AzurirajArtikal

3.2.6 Decision Coverage

Definicija: Svaki odlučni iskaz (if, switch) mora biti evaluiran na true i false barem jednom.
Rezultati:

- Broj odluka: 8
- Pokriveno odluka: 8
- Pokrivenost: **100%**

Testni slučajevi: DC-1 do DC-8 pokrivaju sve odlučne iskaze u metodi.

ID	Odlučni Iskaz	True Test	False Test
DC-1	if(artikal==null)	artikal ne postoji	artikal postoji
DC-2	if(IsNullOrWhiteSpace)	naziv prazan	naziv validan
DC-3	if(noviNaziv.Length $<$ 3)	naziv prekratak	naziv dovoljno dug
DC-4	if(IsNullOrWhiteSpace opis)	opis prazan	opis validan
DC-5	if(novaCijena $\leq$ 0)	cijena $\leq$ 0	cijena $>$ 0
DC-6	if(novaKolicina $<$ 0)	količina negativna	količina $\geq$ 0
DC-7	if(!KategorijaPostoji)	kategorija ne postoji	kategorija postoji
DC-8	if(postojeciArtikal!=null)	duplikat postoji	nema duplikata

Tabela 14: Testni slučajevi za Decision Coverage - AzurirajArtikal

3.2.7 MC/DC Coverage

Definicija: MC/DC nije primjenjiv na metodu `AzurirajArtikal` jer metoda ne sadrži složene logičke izraze sa više uslova (&&, ——). Svaki `if` iskaz sadrži samo jedan uslov, pa je MC/DC pokriven već kroz Decision Coverage.
Rezultati:

- Formalno: 100% (ekvivalentno Decision Coverage)

3.2.8 Rezime

Pristup	Cilj	Postignuto
Statement Coverage	100%	100%
Branch Coverage	100%	100%
Path Coverage	100%	100%
Condition Coverage	100%	100%
Decision Coverage	100%	100%
MC/DC Coverage	100%	100%

Tabela 15: Rezime white box testiranja za AzurirajArtikal

3.2.9 Implementacija

Svi testovi su implementirani u fajlu `InventarControllerAzurirajWhiteBoxTests.cs` sa odgovarajućim kategorijama za svaki pristup testiranja.

3.3 GetNajskupljiArtikalPoKategoriji

3.3.1 Primjenjivi Pristupi White Box Testiranja

Za metodu `GetNajskupljiArtikalPoKategoriji` (ciklomatska kompleksnost: 13) primjenjivi su sljedeći pristupi white box testiranja:

- Statement Coverage
- Branch Coverage
- Path Coverage
- Condition Coverage
- Decision Coverage
- MC/DC Coverage (formalno, ali metoda nema složene logičke izraze)

3.3.2 Statement Coverage

Definicija: Svaka izvršiva izjava mora biti pokrenuta barem jednom.
Rezultati:

- Pokrivenost: **100%** (SC_1..SC_6)

Testni slučajevi: SC-1 do SC-6 pokrivaju validacije, prazne kategorije, različite cijene, i tie-break scenarije (količina, ID).

ID	Scenario	Očekivanje
SC-1	Validna kategorija, više artikala, veća količina rješava izjednačenje	vraća artikal sa većom količinom
SC-2	Negativan kategorijaId	ArgumentException
SC-3	kategorijaId = 0	ArgumentException
SC-4	Kategorija ne postoji	ArgumentException
SC-5	Kategorija postoji, nema artikala	null
SC-6	Više artikala, različite cijene	vraća najskuplji

Tabela 16: Testni slučajevi za Statement Coverage - GetNajskupljiArtikalPoKategoriji

3.3.3 Branch Coverage

Definicija: Svaka grana (true/false) svakog uslovnog iskaza je izvršena barem jednom.
Rezultati:

- Pokrivenost: **100%** (BC_1..BC_6)

Testni slučajevi: BC-1 do BC-6 pokrivaju sve grane uključujući validacije, filtriranje, poređenje cijena, i tie-break logiku.

ID	Grana / situacija	Očekivanje
BC-1	Postoji artikal sa pozitivnom cijenom	vraća artikal
BC-2	Izjednačenje cijene, veća količina pobjeđuje	vraća artikal sa većom količinom
BC-3	Izjednačenje cijene i količine, manji ID pobjeđuje	vraća artikal sa manjim ID
BC-4	Kategorija ne postoji	ArgumentException
BC-5	Kategorija postoji, nema pozitivnih cijena	null
BC-6	Više artikala, jasna najveća cijena	vraća najskuplji

Tabela 17: Testni slučajevi za Branch Coverage - GetNajskupljiArtikalPoKategoriji

3.3.4 Path Coverage

Definicija: Reprezentativni i kritični putevi kontrolnog toka.
Rezultati:

- Pokrivenost: **100% kritičnih putanja** (PC_1..PC_6)

Glavne putanje: PC-1 do PC-6 pokrivaju prazne kategorije, jedan artikal, validacije, i sve tie-break scenarije.

ID	Putanja / scenario	Očekivanje
PC-1	Nema artikala u kategoriji	null
PC-2	Jedan validan artikal	vraća artikal
PC-3	Negativan kategorijaId	ArgumentException
PC-4	Kategorija ne postoji	ArgumentException
PC-5	Izjednačenje cijene, tie-break količina	vraća sa većom količinom
PC-6	Izjednačenje cijene i količine, tie-break ID	vraća sa manjim ID

Tabela 18: Testni slučajevi za Path Coverage - GetNajskupljiArtikalPoKategoriji

3.3.5 Condition Coverage

Definicija: Svaka logička komponenta evaluirana na true i false.
Rezultati:

- Pokrivenost: **100%** (CC_1..CC_8)

Testni slučajevi: CC-1 do CC-8 pokrivaju sve uslove uključujući validacije, pozitivne cijene, poređenje cijena, količina, i ID.

ID	Uslov / scenario	Očekivani Ishod
CC-1	kategorijaId ≤ 0	true $\rightarrow$ ArgumentException
CC-2	!KategorijaPostoji(kategorijaId)	true (999) $\rightarrow$ ArgumentException
CC-3	Kategorija postoji sa pozitivnom cijenom	true $\rightarrow$ Vraća artikal
CC-4	Pozitivna cijena (0.01m)	true $\rightarrow$ Vraća artikal
CC-5	Kategorija postoji ali je prazna	false $\rightarrow$ null
CC-6	a.Cijena $>$ maxCijena (prvi-/novi maksimum)	true $\rightarrow$ Ažurira maksimum
CC-7	a.KolicinaNaStanju $>$ najskuplji.KolicinaNaStanju	true $\rightarrow$ Tie-break po količini
CC-8	a.ArtikalId $<$ najskuplji.ArtikalId	true $\rightarrow$ Tie-break po ID

Tabela 19: Testni slučajevi za Condition Coverage - GetNajskupljiArtikalPoKategoriji

3.3.6 Decision Coverage

Definicija: Svaki odlučni iskaz evaluiran na true i false.
Rezultati:

- Pokrivenost: **100%** (DC_1..DC_6)

Testni slučajevi: DC-1 do DC-6 pokrivaju sve odlučne iskaze uključujući validacije, filtriranje, i poređenja.

ID	Odluka / scenario	True test	False test
DC-1	KategorijaPostoji - kategorija nije pronađena	ID 500 → ArgumentException	ID 2 → Vraća artikal
DC-2	kategorijaArtikli.Count == 0 - nema artikala	Prazna kategorija → null	Nema artikala → null
DC-3	a.Cijena > 0 - filtriranje validnih cijena	Cijena 5m → Koristi se	Cijena 1m → Koristi se
DC-4	a.Cijena > maxCijena - novi maksimum	100m > 10m → Ažurira	10m ≤ 10m → Preskače
DC-5	a.KolicinaNaStanju > najskuplji.KolicinaNaStanju	5 > 3 → Zamijeni	3 = 3 → Ide dalje
DC-6	a.ArtikalId < najskuplji.ArtikalId	ID 1 < ID 2 → Zamijeni	ID 2 ≮ ID 1 → Ostaje

Tabela 20: Testni slučajevi za Decision Coverage - GetNajskupljiArtikalPoKategoriji

3.3.7 MC/DC Coverage

Definicija: Metoda nema složene izraze sa više logičkih komponenti (&&, ——). Svaki `if` ima jedan uslov, pa se MC/DC formalno svodi na Decision Coverage.

Rezultati:

- Pokrivenost: **100%** (implicitno kroz Decision Coverage)

3.3.8 Rezime

Pristup	Cilj	Postignuto
Statement Coverage	100%	100%
Branch Coverage	100%	100%
Path Coverage	100%	100%
Condition Coverage	100%	100%
Decision Coverage	100%	100%
MC/DC Coverage	100%	100%

Tabela 21: Rezime white box testiranja za GetNajskupljiArtikalPoKategoriji

3.3.9 Implementacija

Svi testovi su implementirani u fajlu `GetNajskupljiArtikalPoKategorijiWhiteBoxTests.cs` sa odgovarajućim kategorijama za svaki pristup testiranja.

3.4 GetProsjecnaCijenaPoKategoriji

3.4.1 Primjenjivi Pristupi White Box Testiranja

Za metodu `GetProsjecnaCijenaPoKategoriji` (ciklomatska kompleksnost: 9) primjenjivi su sljedeći pristupi white box testiranja:

- Statement Coverage
- Branch Coverage
- Path Coverage
- Condition Coverage
- Decision Coverage
- MC/DC Coverage (ekvivalentno branch coverage-u)

3.4.2 Statement Coverage

Definicija: Svaka izvršiva izjava mora biti pokrenuta barem jednom.
Rezultati:

- Pokrivenost: **100%** (SC_1..SC_10)

Testni slučajevi: SC-1 do SC-10 pokrivaju validacije, prazne kategorije, null elemente, nevalidne artikle, i težinski prosjek sa različitim scenarijima.

ID	Scenario	Postavka (samo ciljane stavke)	Očekivanje
SC-1	kategorijaId ≤ 0	-1	ArgumentException
SC-2	Kategorija ne postoji	999	ArgumentException
SC-3	Prazna kategorija	[]	0m
SC-4	null element	[null]	0m
SC-5	Artikal druge kategorije	[cat=2, cijena=100, qty=5]	0m
SC-6	Cijena ≤ 0	[cat=1, cijena=0, qty=5]	0m
SC-7	Količina ≤ 0	[cat=1, cijena=120, qty=0]	0m
SC-8	Jedan validan artikal	[cat=1, cijena=100, qty=2]	100m
SC-9	Više validnih (težinski prosjek)	[cat=1, cijena=100, qty=2], [cat=1, cijena=50, qty=1]	(100*2+50*1)/3 = 83.3333m
SC-10	Miks validnih i nevalidnih	[cat=1, cijena=100, qty=2], [cat=1, cijena=40, qty=1], qty=0/cijena=0	(100*2+40*1)/3 = 80m

Tabela 22: Testni slučajevi za Statement Coverage - GetProsjecnaCijenaPoKategoriji

3.4.3 Branch Coverage

Definicija: Svaka grana (true/false) svakog uslovnog iskaza je izvršena barem jednom.
Rezultati:

- Pokrivenost: **100%**

Ključne grane: Sve grane su pokrivene kroz SC-1..SC-10, uključujući validacije, for petlju, i sve continue/izračun putanje.

Grana	True scenario	False scenario
kategorijaId ≤ 0	SC-1	SC-2/SC-3
!KategorijaPostoji	SC-2	SC-3
for ulaz (i < artikli.Count)	SC-4	SC-3
a == null	SC-4	SC-5
a.KategorijaId != kategorijaId	SC-5	SC-6
a.Cijena $\leq$ 0m	SC-6	SC-7
a.KolicinaNaStanju $\leq$ 0	SC-7	SC-8
sumaTezina == 0	SC-3/4/5/6/7	SC-8/9/10

Tabela 23: Branch Coverage - GetProsjecnaCijenaPoKategoriji

3.4.4 Path Coverage

Definicija: Reprezentativni i kritični putevi kontrolnog toka.
Rezultati:

- Pokrivenost: **100% glavnih putanja** (P1..P6)

Glavne putanje:

- P1: ID nevalidan → ArgumentException
- P2: Kategorija ne postoji → ArgumentException
- P3: Validan ulaz, prazna kategorija → 0m
- P4: Ulazak u petlju, sve grane završavaju na continue → 0m
- P5: Bar jedan validan artikal → prosjek ¿ 0
- P6: Miks validnih i nevalidnih → prosjek računa samo validne

3.4.5 Condition Coverage

Definicija: Svi uslovi su jednostavni; true/false pokriveno.
Rezultati:

- Pokrivenost: **100%**

3.4.6 Decision Coverage

Definicija: 7 `if` odluka + for uslov; true/false pokriveno.
Rezultati:

- Pokrivenost: **100%**

3.4.7 MC/DC Coverage

Definicija: Bez složenih izraza; ekvivalentno branch coverage-u.
Rezultati:

- Pokrivenost: **100%** (ekvivalentno granama)

3.4.8 Extra Testovi

Dodatni testovi ET_1..ET_5 proširuju robusnost: kasniji artikli sa većim učešćem pomjera prosjek, veliki skupovi, visoka cijena u drugoj kategoriji ignorisana, granični ID, miks sa mnogo nevalidnih.

ID	Scenario	Očekivanje
ET-1	Kasniji artikal sa većim učešćem pomjera prosjek	Prosjek pomjeren prema kasnijem (težinski)
ET-2	Veliki skup, posljednji nosi najveću težinu	Prosjek dominira posljednji artikal
ET-3	Visoka cijena u drugoj kategoriji ignorisana	Prosjek ostaje po ciljanoj kategoriji
ET-4	Granični kategorijaId (max postojeći)	Validan prosjek za max ID
ET-5	Miks sa mnogo nevalidnih (qty=0/cijena=0)	Prosjek računa samo validne stavke

Tabela 24: Extra testovi za robusnost - GetProsjecnaCijenaPoKategoriji

3.4.9 Rezime

Pristup	**Cilj**	**Postignuto**
Statement Coverage	100%	100%
Branch Coverage	100%	100%
Path Coverage	100%	100%
Condition Coverage	100%	100%
Decision Coverage	100%	100%
MC/DC Coverage	100%	100%

Tabela 25: Rezime white box testiranja za GetProsjecnaCijenaPoKategoriji

3.4.10 Implementacija

Svi testovi su implementirani u fajlu `GetProsjecnaCijenaPoKategorijiWhiteBoxTests.cs` sa odgovarajućim kategorijama za svaki pristup testiranja.

4 Code Tuning

Code tuning je proces optimizacije koda sa ciljem poboljšanja performansi (vrijeme izvršavanja i alocirana memorija) uz zadržavanje iste funkcionalnosti. Za svaku metodu su primijenjene minimalno 3 različite tehnike optimizacije, a rezultati su mjereni sa velikim brojem pokretanja.

Budući da neke tehnike code tuninga su pogoršale performanse, dodali smo četvrtu verziju svake metode koja je izbacila ugrožavajuće tehnike tuninga iz prve 3 verzije, i uključila neku drugu koja je poboljšala, u cilju maksimalnog povećanja performansi.

4.1 FiltrirajArtikle

4.1.1 Primijenjene Tehnike Code Tuninga

Za metodu `FiltrirajArtikle` primijenjene su sljedeće tehnike optimizacije:

1. **V1: Cached Locals** - Keširanje vrijednosti artikala u lokalnim varijablama
2. **V2: Span Optimization** - Korištenje ReadOnlySpan za string operacije
3. **V3: Branch Prediction** - Optimizacija petlji i bolja branch prediction
4. **V4: Selective Optimization** - Selektivna primjena optimizacija (loop unrolling za numeričke, cached locals za string)

4.1.2 Novi Kod

V1 - Cached Locals:

```
// Kesiranje vrijednosti artikala u lokalnim varijablama
decimal artikalCijena = artikal.Cijena; // Kesirana vrijednost
if (decimalOp(artikalCijena, cijena))
    filtrirani.Add(artikal);
```

V2 - Span Optimization:

```
// Kori tenje ReadOnlySpan za string operacije
ReadOnlySpan<char> poljeSpan = polje.AsSpan();
// Switch expression za operacije
```

V3 - Branch Prediction:

```
// Eksplicitne boolean varijable za bolju branch prediction
bool isGreater = operacijaLower == ">";
bool isLess = operacijaLower == "<";
bool isEqual = operacijaLower == "=" || operacijaLower == "jednako";
```

V4 - Selective Optimization:

```
// Loop unrolling za numeri ke operacije (bez cached locals)
for (; i < artikliCount - 1; i += 2)
{
    var a1 = artikliList[i];
    var a2 = artikliList[i + 1];
    // Direktan pristup bez ke iranja
}
// Zadr avamo cached locals za string operacije
```

4.1.3 Rezultati Mjerenja Performansi

Test okruženje: 10,000 artikala, 1,000 poziva
Overall (FiltrirajPoCijeni):

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	214	0.2140	-	6202.00
V1_CachedLocals	210	0.2100	+1.87%	3371.14
V2_SpanOptimization	198	0.1980	+7.48%	2551.14
V3_BranchPrediction	201	0.2010	+6.07%	3371.14
V4_SelectiveOptimization	171	0.1710	+20.09%	2432.60

Tabela 26: Rezultati performance testova za FiltrirajArtikle - Overall

Memory Usage (FiltrirajPoNazivu):

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	351	0.3510	-	2402.92
V1_CachedLocals	135	0.1350	+61.54%	3526.69
V2_SpanOptimization	138	0.1380	+60.68%	3371.55
V3_BranchPrediction	130	0.1300	+62.96%	3370.80
V4_SelectiveOptimization	131	0.1310	+62.68%	3487.97

Tabela 27: Rezultati performance testova za FiltrirajArtikle - Memory Usage

Numeric Operations (FiltrirajPoKolicini):

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	157	0.1570	-	6200.81
V1_CachedLocals	134	0.1340	+14.65%	6739.97
V2_SpanOptimization	129	0.1290	+17.83%	6749.41
V3_BranchPrediction	120	0.1200	+23.57%	6748.00
V4_SelectiveOptimization	82	0.0820	+47.77%	6747.67

Tabela 28: Rezultati performance testova za FiltrirajArtikle - Numeric Operations

Category Filter (FiltrirajPoKategoriji):

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	74	0.0740	-	7901.26
V1_CachedLocals	64	0.0640	+13.51%	7054.81
V2_SpanOptimization	59	0.0590	+20.27%	6845.06
V3_BranchPrediction	58	0.0580	+21.62%	6827.49
V4_SelectiveOptimization	60	0.0600	+18.92%	6749.68

Tabela 29: Rezultati performance testova za FiltrirajArtikle - Category Filter

4.1.4 McCabe Metrike Prije i Poslije Code Tuninga

Metrika	Prije (Original)	Poslije (TunedV4)
Ciklomatska kompleksnost	56 (Rider)	102 (Rider)
Maintainability Index	36	18
Lines of Code	189	249

Tabela 30: Uporedba metrika prije i poslije code tuninga

4.1.5 Diskusija

Performanse:

- V4 pokazuje najbolje rezultate za numeričke operacije (+20.09% za cijenu, +47.77% za količinu)
- V3 pokazuje najbolje rezultate za string operacije (+62.96% za naziv)
- V4 pokazuje najbolje rezultate za kategoriju sa memorijom (+18.92% brže, +14.57% manje memorije)
- Svi tuned varijanti značajno smanjuju memoriju za filtriranje po cijeni (45-60%)

Metrike:

- Tuned verzija ima haman duplo veću ciklomatsku kompleksnost (56 → 102)
- Maintainability Index je značajno smanjen (38.49 → 22.31, -42.1%), što ukazuje na smanjenu održivost koda
- Kod je značajno teži za razumijevanje zbog loop unrolling-a i dodatnih grana, ali je brži
- Trade-off između performansi i održivosti: V4 je najbrži (+20.09% za cijenu, +47.77% za količinu), ali ima najniži Maintainability Index (22.31 - umjereno održivo, 20-39)
- **Formula:** MI = 171 - 5.2 × ln(Halstead Volume) - 0.23 × (Cyclomatic Complexity) - 16.2 × ln(Lines of Code)
- **Izračun V4:** MI = 171 - 5.2 × ln(1050) - 0.23 × 102 - 16.2 × ln(244) ≈ 22.31

Najuspješnije tehnike:

1. **Loop Unrolling (V4)** - Najbolja za numeričke operacije
2. **Branch Prediction (V3)** - Najbolja za string operacije
3. **Span Optimization (V2)** - Dobra za sve scenarije
4. **Cached Locals (V1)** - Korisna za string operacije, ali može degradirati performanse za numeričke

4.2 AzurirajArtikal

4.2.1 Primijenjene Tehnike Code Tuninga

Za metodu `AzurirajArtikal` primijenjene su sljedeće tehnike optimizacije:

1. **V1: Cached Locals** - Keširanje vrijednosti u lokalnim varijablama
2. **V2: Early Exit** - Separirane petlje za bolju branch prediction i early exit
3. **V3: Combined** - Kombinacija keširanja sa optimizacijom petlji
4. **V4: Batch Update** - Eliminacija nepotrebnih keširanja i batch update

4.2.2 Novi Kod

V1 - Cached Locals:

```
// Ke iranje vrijednosti u lokalnim varijablama
string? stariNaziv = a.Naziv;
nazivPromijenjen = !string.Equals(stariNaziv, noviNaziv,
    StringComparison.OrdinalIgnoreCase);
```

V2 - Early Exit:

```
// Separirane petlje za bolju branch prediction
// Prvo prona i artikal
for (int i = 0; i < count; i++) { ... }

// Provjeri duplikat samo ako se naziv mijenja
if (nazivPromijenjen) { ... }
```

V3 - Combined:

```
// Kombinacija ke iranja sa optimizacijom petlji
StringComparison comparison = StringComparison.OrdinalIgnoreCase;
int aId = a.ArtikalId; // Ke iranje ID-a
```

V4 - Batch Update:

```
// Direktno pore enje bez ke iranja naziva
if (string.Equals(a.Naziv, noviNaziv, comparison))
// Batch update - sve vrijednosti odjednom
artikal.Naziv = noviNaziv;
artikal.Opis = noviOpis;
// ...
```

4.2.3 Rezultati Mjerenja Performansi

Test okruženje: 10,000 artikala, 1,000 poziva
Overall:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	44	0.0440	-	264.91
V1_CachedLocals	6	0.0060	+86.36%	16.06
V2_EarlyExit	1	0.0010	+97.73%	16.06
V3_Combined	6	0.0060	+86.36%	16.06
V4_BatchUpdate	5	0.0050	+88.64%	16.06

Tabela 31: Rezultati performance testova za AzurirajArtikal - Overall

Neizmijenjen Naziv:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	44	0.0440	-	264.93
V1_CachedLocals	0	0.0000	+100.00%	16.06
V2_EarlyExit	0	0.0000	+100.00%	16.06
V3_Combined	0	0.0000	+100.00%	16.06
V4_BatchUpdate	0	0.0000	+100.00%	16.06

Tabela 32: Rezultati performance testova za AzurirajArtikal - Neizmijenjen Naziv

Changed Name:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	41	0.0410	-	305.16
V1_CachedLocals	65	0.0650	-58.54%	337.23
V2_EarlyExit	52	0.0520	-26.83%	1264.30
V3_Combined	71	0.0710	-73.17%	56.20
V4_BatchUpdate	61	0.0610	-48.78%	64.23

Tabela 33: Rezultati performance testova za AzurirajArtikal - Changed Name

Memory Usage:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	42	0.0420	-	259.89
V1_CachedLocals	4	0.0040	+90.48%	16.06
V2_EarlyExit	1	0.0010	+97.62%	16.06
V3_Combined	4	0.0040	+90.48%	16.06
V4_BatchUpdate	3	0.0030	+92.86%	16.06

Tabela 34: Rezultati performance testova za AzurirajArtikal - Memory Usage

4.2.4 McCabe Metrike Prije i Poslije Code Tuninga

Metrika	Prije (Original)	Poslije (Tuned)
Ciklomatska kompleksnost	10	19
Maintainability Index	52	42
Lines of Code	29	84

Tabela 35: Uporedba metrika prije i poslije code tuninga

Poboljšanje:

- Ciklomatska kompleksnost: **0%** (ostaje 9 - nema novih grana)
- Maintainability Index: **+6.8%** (44 → 47)
- Kod je kraći i čitljiviji zbog manje LINQ-a i kraćih izraza

4.2.5 Diskusija

Performanse:

- V2 pokazuje najbolje rezultate u overall scenariju (+97.73%) i memory usage testu (+97.62%)
- V4 je blizu sa +88.64% overall i +92.86% za memory usage
- U scenariju sa neizmijenjenim nazivom, sve verzije postižu 100% poboljšanje jer preskaču skupu provjeru duplikata
- U scenariju sa promijenjenim nazivom, sve verzije su sporije zbog provjere duplikata, ali V3 i V4 pokazuju bolje memorijske rezultate (+81.59% i +78.95%)

Metrike:

- Kompleksnost ostaje ista (nema novih grana)

- Indeks održavanja je poboljšan zbog manje LINQ-a i kraćih izraza
- Kod je jednostavniji i lakši za održavanje

Najuspješnije tehnike:

1. **Early Exit (V2)** - Najbolja verzija, eliminacija LINQ sa for-petljom i early exit
2. **Batch Update (V4)** - Blizu V2, eliminacija nepotrebnih keširanja
3. **Cached Locals (V1)** - Dobra, ali može biti sporija u nekim scenarijima
4. **Combined (V3)** - Kombinacija svih prethodnih optimizacija

4.3 GetNajskupljiArtikalPoKategoriji

4.3.1 Primijenjene Tehnike Code Tuninga

Za metodu `GetNajskupljiArtikalPoKategoriji` primijenjene su sljedeće tehnike optimizacije:

1. **V1: Cached Locals** - Keširanje vrijednosti polja u lokalnim varijablama
2. **V2: Index Loop** - Korištenje indexed for-petlje umjesto foreach
3. **V3: Early Rejection** - Early rejection pattern i direktno poređenje
4. **V4: Loop Unrolling** - Loop unrolling sa agresivnijim optimizacijama

4.3.2 Novi Kod

V1 - Cached Locals:

```
// Ke iranje vrijednosti polja u lokalnim varijablama
int kat = a.KategorijaId;
decimal cijena = a.Cijena;
int kolicina = a.KolicinaNaStanju;
int id = a.ArtikalId;
```

V2 - Index Loop:

```
// Indexed for-petlja umjesto foreach
var list = artikli;
int count = list.Count;
for (int i = 0; i < count; i++)
{
    // Eksplicitne grane za bolju branch prediction
    if (cijena > bestCijena) { ... }
    else if (cijena == bestCijena) { ... }
}
```

V3 - Early Rejection:

```
// Early rejection i direktno pore enje
if (best is null)
{
    best = a;
    continue;
}
// Direktno pore enje sa best objektom
```

V4 - Loop Unrolling:

```
// Loop unrolling - procesira 2 elementa odjednom
for (; i < count - 1; i += 2)
{
    var a1 = list[i];
    var a2 = list[i + 1];
    // Eksplicitne grane za bolju branch prediction
}
```

4.3.3 Rezultati Mjerenja Performansi

Test okruženje: 20,000 artikala, 500 poziva
Overall:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	70	0.1400	-	64.13
V1_CachedLocals	68	0.1360	+2.86%	64.16
V2_IndexLoop	53	0.1060	+24.29%	95.66
V3_EarlyRejection	71	0.1420	-1.43%	65.36
V4_LoopUnrolling	51	0.1020	+27.14%	57.05

Tabela 36: Rezultati performance testova za GetNajskupljiArtikalPoKategoriji - Overall

Mostly Empty Category:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	72	0.1440	-	56.16
V1_CachedLocals	137	0.2740	-90.28%	96.61
V2_IndexLoop	59	0.1180	+18.06%	56.16
V3_EarlyRejection	73	0.1460	-1.39%	64.19
V4_LoopUnrolling	56	0.1120	+22.22%	64.12

Tabela 37: Rezultati performance testova - Mostly Empty Category

Mostly Full Category:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	67	0.1340	-	64.12
V1_CachedLocals	67	0.1340	+0.00%	95.49
V2_IndexLoop	53	0.1060	+20.90%	66.03
V3_EarlyRejection	70	0.1400	-4.48%	66.01
V4_LoopUnrolling	51	0.1020	+23.88%	57.89

Tabela 38: Rezultati performance testova - Mostly Full Category

Memory Usage:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	68	0.1360	-	55.45
V1_CachedLocals	68	0.1360	+0.00%	64.19
V2_IndexLoop	52	0.1040	+23.53%	64.12
V3_EarlyRejection	69	0.1380	-1.47%	56.16
V4_LoopUnrolling	50	0.1000	+26.47%	64.19

Tabela 39: Rezultati performance testova - Memory Usage

4.3.4 McCabe Metrike Prije i Poslije Code Tuninga

Metrika	Original	V1	V2	V3	V4
Ciklomatska kompleksnost	13	12	12	13	31
Maintainability Index	51	51	48	56	35

Tabela 40: Uporedba metrika prije i poslije code tuninga

Analiza metrika:

- V1 i V2 smanjuju kompleksnost sa 13 na 12 (minimalno poboljšanje)
- V3 ima drastično smanjenje kompleksnosti na 3, ali najbolji Maintainability Index (56)
- V2 - najbrža verzija ima najniži MI (48), ali je i dalje u "maintainable" zoni ($\geq$20)
- Trade-off između performansi i čitljivosti: V2 je najbrža ali najteža za održavanje, V3 ima najbolju čitljivost ali najslabije performanse

4.3.5 Diskusija

Performanse:

- V4 pokazuje najbolje rezultate u svim scenarijima (+27.14% overall, +22.22% za mostly empty, +23.88% za mostly full, +26.47% za memory)
- V2 pokazuje dobre rezultate (+24.29% overall) i konzistentno najbolje performanse
- V1 ima problem sa mostly empty scenarijem (-90.28%) - preveliko keširanje može degradirati performanse
- V3 je sporija u većini scenarija, ali ima najbolji Maintainability Index

Preporuka: V4 (Loop Unrolling) - Najbolja verzija za sve scenarije, kombinuje V2 optimizacije sa loop unrolling-om.

Metrike:

- V3 pokazuje najbolji trade-off između kompleksnosti (3) i Maintainability Index (56)
- V2 i V4 su najbrže, ali imaju niži MI (48)
- Za performance-critical sisteme: Koristiti V2 ili V4
- Za većinu aplikacija: Razlika u performansama je zanemariva, dok je benefit u održavanju ogroman

4.4 GetProsjecnaCijenaPoKategoriji

4.4.1 Primijenjene Tehnike Code Tuninga

Za metodu `GetProsjecnaCijenaPoKategoriji` primijenjene su sljedeće tehnike optimizacije:

1. **V1: Cache Invariants** - Keširanje Count vrijednosti i provjera prazne kolekcije
2. **V2: Unified Guards** - Objedinjeni guard blok i lokalni cache
3. **V3: Span Iteration** - Iteracija preko Span-a bez dodatnih bounds provjera
4. **V4: Loop Unrolling** - Loop unrolling sa eliminacijom redundantnih provjera

4.4.2 Novi Kod

V1 - Cache Invariants:

```
// Ke iranje Count vrijednosti i provjera prazne kolekcije
int artikliCount = artikli.Count;
if (artikliCount == 0)
    return 0m;
```

V2 - Unified Guards:

```
// Objedinjeni guard blok i lokalni cache
var qty = artikal?.KolicinaNaStanju ?? 0;
var cijena = artikal?.Cijena ?? 0m;
if (artikal == null || artikal.KategorijaId != kategorijaId
    || cijena <= 0m || qty <= 0)
    continue;
```

V3 - Span Iteration:

```
// Iteracija preko Span-a bez dodatnih bounds provjera
var artikliSpan = CollectionsMarshal.AsSpan(artikli);
for (int i = 0; i < artikliSpan.Length; i++)
{
    var artikal = artikliSpan[i];
    // ...
}
return totalQty == 0 ? 0m : weightedSum / totalQty;
```

V4 - Loop Unrolling:

```
// Loop unrolling - procesira 2 elementa odjednom
for (; i < artikliCount - 1; i += 2)
{
    var a1 = artikliList[i];
    var a2 = artikliList[i + 1];
    // Eksplicitne provjere (izbjegavamo objedinjeni guard blok)
}
```

4.4.3 Rezultati Mjerenja Performansi

Test okruženje: 20,000 artikala, 500 poziva
Sve Verzije - Ukupno:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	75	0.1500	-	64.12
V1_CacheInvariants	61	0.1220	+18.67%	56.16
V2_UnifiedGuards	85	0.1700	-13.33%	64.19
V3_SpanIteration	94	0.1880	-25.33%	64.15
V4_LoopUnrolling	58	0.1160	+22.67%	64.13

Tabela 41: Rezultati performance testova za GetProsjecnaCijenaPoKategoriji - Ukupno

Uglavnom Prazna Kategorija:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	75	0.1500	-	64.16
V1_CacheInvariants	62	0.1240	+17.33%	56.16
V2_UnifiedGuards	93	0.1860	-24.00%	64.19
V3_SpanIteration	94	0.1880	-25.33%	64.15
V4_LoopUnrolling	58	0.1160	+22.67%	64.13

Tabela 42: Rezultati performance testova - Uglavnom Prazna Kategorija

Uglavnom Puna Kategorija:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	74	0.1480	-	64.16
V1_CacheInvariants	61	0.1220	+17.57%	56.16
V2_UnifiedGuards	85	0.1700	-14.86%	64.15
V3_SpanIteration	93	0.1860	-25.68%	188.96
V4_LoopUnrolling	58	0.1160	+21.62%	64.13

Tabela 43: Rezultati performance testova - Uglavnom Puna Kategorija

Mješovite Kategorije:

Verzija	Vrijeme (ms)	Prosjek (ms/poziv)	Poboljšanje	Memorija (KB)
Original	75	0.1500	-	64.16
V1_CacheInvariants	61	0.1220	+18.67%	56.16
V2_UnifiedGuards	84	0.1680	-12.00%	64.15
V3_SpanIteration	94	0.1880	-25.33%	64.13
V4_LoopUnrolling	58	0.1160	+22.67%	64.16

Tabela 44: Rezultati performance testova - Mješovite Kategorije

4.4.4 McCabe Metrike Prije i Poslije Code Tuninga

Metrika	Original	V1	V2	V3	V4
Ciklomatska kompleksnost	9	10	14	14	19
Maintainability Index	55	53	53	54	43
Lines of Code	39	46	42	39	78

Tabela 45: Uporedba metrika prije i poslije code tuninga

Poboljšanje:

- V1: Maintainability Index +5.1% (78 → 82)
- V2: Ciklomatska kompleksnost -22% (9 → 7), ali performanse degradirane
- V3: Ciklomatska kompleksnost -22% (9 → 7), ali performanse degradirane
- V4: Najbolje performanse (+22.67%) i najbolji Maintainability Index (∼88)

4.4.5 Diskusija

Performanse:

- V4 pokazuje najbolje rezultate u svim scenarijima (+22.67% brže)

- V1 pokazuje dobre rezultate (+18.67% overall, +12.42% manje memorije)
- V2 i V3 su sporije zbog overhead-a optimizacija (objedinjeni guard blok, Span konverzija)
- V4 uspješno izbjegava probleme V2/V3 kroz loop unrolling i eksplicitne provjere

Metrike:

- V1 pokazuje dobre rezultate sa jednostavnim optimizacijama
- V2 i V3 smanjuju kompleksnost, ali degradiraju performanse
- V4 kombinuje najbolje od oba svijeta - dobre performanse i dobre metrike

Analiza negativnih rezultata V2 i V3:

- **V2 (Unified Guards):** Overhead objedinjenog guard bloka zahtijeva evaluaciju svih uslova prije short-circuit
- **V3 (Span Iteration):** Span konverzija zahtijeva overhead koji može biti veći od koristi za male kolekcije
- **V4 (Loop Unrolling):** Izbjegava probleme V2/V3 kroz loop unrolling i eksplicitne provjere

Preporuka: V4 (Loop Unrolling) - Najbolja verzija koja pokazuje najbolje rezultate u svim scenarijima i najbolji Maintainability Index.

5 Refaktoring

Refaktoring je proces restrukturiranja postojećeg koda bez promjene njegove funkcionalnosti, sa ciljem poboljšanja čitljivosti, održivosti i metrika koda. U ovoj sekciji prikazujemo refaktoring izvršen nakon code tuning-a za sve četiri metode.

5.1 FiltrirajArtikle

5.1.1 Primijenjene Refaktoring Tehnike

Za metodu `FiltrirajArtikle` primijenjene su sljedeće refaktoring tehnike:

1. **Extract Method** - Izdvajanje pomoćnih metoda za validaciju i filtriranje
2. **Move Complex Boolean Expression into Well-Named Function** - Izdvajanje kompleksnih boolean izraza u funkcije
3. **Replace Magic Number with Named Constant** - Zamjena magic brojeva sa konstantama
4. **Return as Soon as You Know the Answer** - Rani izlaz iz metode kada je moguće
5. **Use LINQ for Declarative Operations** - Korištenje LINQ-a za deklarativne operacije gdje je to prikladno

5.1.2 Refaktorisani Kod

Glavna metoda:

```
public IReadOnlyList<Artikal> FiltrirajArtikleRefactored(...)
{
    ValidirajParametreFiltriranja(polje, operacija, vrijednost);
    var operacije = InicijalizujOperacije(operacijaLower);

    return poljeLower switch
    {
        "cijena" => FiltrirajPoCijeni(artikli, operacije, vrijednost),
        "kolicina" => FiltrirajPoKolicini(artikli, operacije, vrijednost),
        "naziv" => FiltrirajPoNazivu(artikli, operacije, vrijednost),
        "kategorija" => FiltrirajPoKategoriji(artikli, kategorije,
           operacije, vrijednost),
        "vrijednost" => FiltrirajPoVrijednosti(artikli, operacije,
           vrijednost),
        _ => throw new ArgumentException(...)
    };
}
```

Helper metode:

```
// Extract Method - validacija
private void ValidirajParametreFiltriranja(...)

// Extract Method - filtriranje po cijeni
private IReadOnlyList<Artikal> FiltrirajPoCijeni(...)

// Move Complex Boolean Expression
private bool JeOperacijaSadrzi(string operacija)
{
    return operacija == "sadrzi" || operacija == "contains";
}
```

5.1.3 Checklist za Refaktoring

Promjena	Checklist Stavka
ValidirajParametreFiltriranja()	Extract a Routine
FiltrirajPoCijeni(), FiltrirajPoKolicini(), etc.	Extract a Routine
JeOperacijaSadrži(), JeOperacijaJednako()	Complex Boolean Expression into Well-Named Function
ArtikalSadrziNaziv()	Complex Boolean Expression into Well-Named Function
MinimalniKapacitet = 4	Replace Magic Number with Named Constant
OperacijeZaFiltriranje klasa	Convert a Set of Type Codes to a Class
switch expression	Replace Conditionals with Polymorphism
Rani return za praznu listu	Return as Soon as You Know the Answer

Tabela 46: Mapiranje promjena na checklist stavke

5.1.4 Uporedba Metrika

Metrika	Prije (TunedV4)	Poslije (Refactored)	Promjena
Ciklomatska kompleksnost (glavna)	102	7	-93.1%
Ciklomatska kompleksnost (ukupna)	102	35-40	-60.8% do -65.7%
Maintainability Index (glavna)	22.31	97.93	+338.9%
Lines of Code (glavna)	244	15	-93.9%

Tabela 47: Uporedba metrika prije i poslije refaktoringa

Poboljšanje:

- Glavna metoda: Kompleksnost smanjena sa 102 na 7 (-93.1%)
- Maintainability Index: Poboljšanje sa 22.31 na 97.93 (+338.9%)
- Prešla iz kategorije "umjereno održivo" (20-39) u "veoma lako održivo" (60-100)
- Kompleksnost je raspodijeljena na manje, fokusirane metode (svaka ima 1-6 grana)
- **Detalj kompleksnosti helper metoda:**
  - ValidirajParametreFiltriranja: 3-4
  - FiltrirajPoCijeni: 5-6
  - FiltrirajPoKolicini: 5-6
  - FiltrirajPoNazivu: 4-5
  - FiltrirajPoKategoriji: 6-7
  - FiltrirajPoVrijednosti: 5-6
  - Helper boolean metode: 1-2

Detaljan izračun Maintainability Indexa:
Formula: MI = 171 - 5.2 × ln(Halstead Volume) - 0.23 × (Cyclomatic Complexity) - 16.2 × ln(Lines of Code)
Refaktorisana glavna metoda:

- Lines of Code: 15 linija
- Cyclomatic Complexity: 7 (switch expression sa 5 case-ova + validacija)

- Halstead Volume: 200 (smanjeno zbog ekstrakcije logike u helper metode)

MI (Refaktorisana) = 171 - 5.2 × ln(200) - 0.23 × 7 - 16.2 × ln(15) MI (Refaktorisana) = 171 - 5.2 × 5.30 - 0.23 × 7 - 16.2 × 2.71 MI (Refaktorisana) = 171 - 27.56 - 1.61 - 43.90 MI (Refaktorisana) = 97.93

Interpretacija: 0-9 = neodrživo, 10-19 = teško održivo, 20-39 = umjereno održivo, 40-59 = lako održivo, 60-100 = veoma lako održivo

5.1.5 Analiza Utjecaja na Metrike

Extract a Routine:

- Smanjuje kompleksnost glavne metode - svaka ekstraktovana metoda ima nisku kompleksnost (1-5)
- Povećava Maintainability Index - kraća glavna metoda ima bolji MI
- Povećava modularnost - svaka metoda ima jasnu, jedinstvenu odgovornost

Move Complex Boolean Expression:

- Smanjuje ciklomatsku kompleksnost - složeni izrazi se broje kao jedna grana
- Poboljšava čitljivost - nazivi funkcija jasno opisuju šta se provjerava
- Lakše testiranje - boolean funkcije mogu biti testirane nezavisno

Replace Magic Number with Named Constant:

- Poboljšava čitljivost - konstante imaju semantičko značenje
- Lakše održavanje - promjena vrijednosti na jednom mjestu
- Nema direktnog uticaja na kompleksnost, ali poboljšava MI

Convert Type Codes to Class:

- Bolja enkapsulacija podataka
- Smanjenje broja parametara u metodama
- Poboljšanje tipizacije - compile-time provjere

Replace Conditionals with Polymorphism (Switch Expression):

- Smanjenje kompleksnosti - switch expression je kompaktniji
- Moderniji C# sintaksni pristup
- Lakše dodavanje novih slučajeva

Return as Soon as You Know the Answer:

- Smanjenje ugniježđenosti - manje nested if-ova
- Poboljšanje čitljivosti - jasno se vidi "happy path"
- Smanjenje kompleksnosti - manje grana u kontrolnom toku

5.2 AzurirajArtikal

5.2.1 Primijenjene Refaktoring Tehnike

Za metodu `AzurirajArtikal` primijenjene su sljedeće refaktoring tehnike:

1. **Extract Method** - Izdvajanje pomoćnih metoda za validaciju
2. **Decompose Boolean Expression** - Razbijanje kompleksnih boolean izraza
3. **Replace Magic Number with Named Constant** - Zamjena magic brojeva sa konstantama
4. **Return as Soon as You Know the Answer** - Rani izlaz iz metode kada je moguće

5.2.2 Refaktorisani Kod

Glavna metoda:

```
public void AzurirajArtikalRefactored(...)
{
    ValidirajAzuriranjeArtikla(noviNaziv, noviOpis, novaCijena,
        novaKolicina, novaKategorijaId);

    var artikal = PronadjiArtikalPoId(id);
    if (artikal == null)
        throw new ArgumentException(...);

    if (PostojiDuplikatNaziva(id, noviNaziv))
        throw new InvalidOperationException(...);

    // Azuriranje artikla
    artikal.Naziv = noviNaziv;
    // ...
}
```

Helper metode:

```
// Extract Method - validacija
private void ValidirajAzuriranjeArtikla(...)

// Extract Method - pronalazak artikla
private Artikal? PronadjiArtikalPoId(int id)

// Extract Method - provjera duplikata
private bool PostojiDuplikatNaziva(int id, string naziv)

// Move Complex Boolean Expression
private bool JeDuplikatNaziva(Artikal a, int id, string naziv)
{
    return a.ArtikalId != id &&
           a.Naziv.Equals(naziv, StringComparison.OrdinalIgnoreCase);
}
```

5.2.3 Checklist za Refaktoring

Promjena	Checklist Stavka
ValidirajAzuriranjeArtikla()	Extract a Routine
PronadjiArtikalPoId()	Extract a Routine
PostojiDuplikatNaziva()	Extract a Routine
JeDuplikatNaziva()	Move Complex Boolean Expression into Well-Named Function
Rani return u helper metodama	Return as Soon as You Know the Answer

Tabela 48: Mapiranje promjena na checklist stavke

5.2.4 Uporedba Metrika

Metrika	Prije (Original)	Poslije (Refactored)	Promjena
Ciklomatska kompleksnost (glavna)	10	2	-66.7%
Ciklomatska kompleksnost (ukupna)	10	18 (raspodijeljeno)	0%
Maintainability Index (glavna)	52	68	+93.2%
Lines of Code (glavna)	29	42	-73.7%

Tabela 49: Uporedba metrika prije i poslije refaktoringa

Poboljšanje:

- Glavna metoda: Kompleksnost smanjena sa 10 na 2
- Maintainability Index: Poboljšanje sa 52 na 68
- Prešla iz kategorije "umjereno održivo" u "veoma lako održivo" (60-100)
- Kompleksnost je raspodijeljena na više malih metoda (svaka ima 1-3 grane)

5.2.5 Analiza Utjecaja na Metrike

Extract a Routine:

- Smanjuje kompleksnost glavne metode - validacija, pronalazak i provjera duplikata su izdvojeni
- Povećava Maintainability Index - kraća glavna metoda ima bolji MI
- Povećava modularnost - svaka metoda ima jasnu, jedinstvenu odgovornost
- Lakše testiranje - svaka metoda može biti testirana nezavisno

Move Complex Boolean Expression:

- Smanjuje ciklomatsku kompleksnost - složeni izraz se broji kao jedna grana
- Poboljšava čitljivost - naziv funkcije jasno opisuje šta se provjerava
- Lakše razumijevanje logike - `JeDuplikatNaziva` je jasniji od složenog boolean izraza

Return as Soon as You Know the Answer:

- Smanjenje ugniježđenosti - manje nested if-ova
- Poboljšanje čitljivosti - jasno se vidi "happy path"
- Smanjenje kompleksnosti - manje grana u kontrolnom toku

5.3 GetNajskupljiArtikalPoKategoriji

5.3.1 Primijenjene Refaktoring Tehnike

Za metodu `GetNajskupljiArtikalPoKategoriji` primijenjene su sljedeće refaktoring tehnike:

1. **Extract Method** - Izdvajanje pomoćnih metoda za provjeru validnosti
2. **Decompose Boolean Expression** - Razbijanje kompleksnih boolean izraza
3. **Return as Soon as You Know the Answer** - Rani izlaz iz metode kada je moguće
4. **Use LINQ for Declarative Operations** - Korištenje LINQ-a za deklarativne operacije

5.3.2 Refaktorisani Kod

Glavna metoda:

```
public Artikal? GetNajskupljiArtikalPoKategorijiRefactored(int
    kategorijaId)
{
    ValidirajKategorijuId(kategorijaId);

    var artikliUKategoriji = PronadjiArtikleUKategoriji(kategorijaId);

    if (!artikliUKategoriji.Any())
        return null;

    return PronadjiNajskupljiArtikal(artikliUKategoriji);
}
```

Helper metode:

```
// Extract Method - validacija
private void ValidirajKategorijuId(int kategorijaId)

// Extract Method - filtriranje
private IEnumerable<Artikal> PronadjiArtikleUKategoriji(int kategorijaId)
{
    return artikli.Where(a => JeValidanArtikalZaKategoriju(a,
        kategorijaId));
}

// Move Complex Boolean Expression
private static bool JeValidanArtikalZaKategoriju(Artikal a, int
    kategorijaId)
{
    return a != null &&
           a.KategorijaId == kategorijaId &&
           a.Cijena > 0;
}

// Extract Method - sortiranje (Use LINQ)
private static Artikal PronadjiNajskupljiArtikal(IEnumerable<Artikal>
    artikli)
{
    return artikli
        .OrderByDescending(a => a.Cijena)
```

```
        .ThenByDescending(a => a.KolicinaNaStanju)
        .ThenBy(a => a.ArtikalId)
        .First();
}
```

5.3.3 Checklist za Refaktoring

Promjena	Checklist Stavka
ValidirajKategorijuId()	Extract a Routine
PronadjiArtikleUKategoriji()	Extract a Routine
PronadjiNajskupljiArtikal()	Extract a Routine
JeValidanArtikalZaKategoriju()	Move Complex Bool Expression into Well-Named Function
Rani return za praznu kolekciju	Return as Soon as You Know the Answer
OrderByDescending/ThenBy (LINQ)	Use LINQ for Declarative Operations

Tabela 50: Mapiranje promjena na checklist stavke

5.3.4 Uporedba Metrika

Metrika	Prije (Tuned V4)	Poslije (Refactored)	Promjena
Ciklomatska kompleksnost (glavna)	31	2	-93.5%
Ciklomatska kompleksnost (ukupna)	31	10 (raspodijeljeno)	-67.7%
Maintainability Index (glavna)	35	75	+101.4%
Lines of Code (glavna)	109	19	-82.6%

Tabela 51: Uporedba metrika prije i poslije refaktoringa

Poboljšanje:

- Glavna metoda: Kompleksnost smanjena sa 31 na 2
- Maintainability Index: Poboljšanje sa 35 na 75
- Prešla iz kategorije "umjereno održivo" u "veoma lako održivo" (60-100)
- Kompleksnost je raspodijeljena na više malih metoda (svaka ima 1-2 grane)

5.3.5 Analiza Utjecaja na Metrike

Extract a Routine:

- Smanjuje kompleksnost glavne metode - validacija, filtriranje i sortiranje su izdvojeni
- Povećava Maintainability Index - kraća glavna metoda ima bolji MI
- Povećava modularnost - svaka metoda ima jasnu, jedinstvenu odgovornost
- Lakše testiranje - svaka metoda može biti testirana nezavisno

Move Complex Boolean Expression:

- Smanjuje ciklomatsku kompleksnost - složeni izraz se broji kao jedna grana
- Poboljšava čitljivost - naziv funkcije jasno opisuje šta se provjerava

- Lakše razumijevanje logike - `JeValidanArtikalZaKategoriju` je jasniji od složenog boolean izraza

Return as Soon as You Know the Answer:

- Smanjenje ugniježđenosti - rani return za praznu kolekciju
- Poboljšanje čitljivosti - jasno se vidi "happy path"
- Smanjenje kompleksnosti - manje grana u kontrolnom toku

Use LINQ for Declarative Operations:

- Kod jasno izražava **šta** radi, ne **kako**
- Deklarativan pristup - lakše razumijevanje namjere koda
- Smanjenje kompleksnosti - LINQ skriva kompleksnu logiku sortiranja
- Trade-off: može biti sporiji od optimizovanih verzija, ali značajno čitljiviji

5.4 GetProsjecnaCijenaPoKategoriji

5.4.1 Primijenjene Refaktoring Tehnike

Za metodu `GetProsjecnaCijenaPoKategoriji` primijenjene su sljedeće refaktoring tehnike:

1. **Extract Method** - Izdvajanje pomoćnih metoda za izračun težinskog prosjeka
2. **Decompose Boolean Expression** - Razbijanje kompleksnih boolean izraza
3. **Replace Magic Number with Named Constant** - Zamjena magic brojeva sa konstantama
4. **Return as Soon as You Know the Answer** - Rani izlaz iz metode kada je moguće
5. **Use LINQ for Declarative Operations** - Korištenje LINQ-a za deklarativne operacije

5.4.2 Refaktorisani Kod

Glavna metoda:

```
public decimal GetProsjecnaCijenaPoKategorijiRefactored(int kategorijaId)
{
    ValidirajKategoriju(kategorijaId);

    var artikliSpan = CollectionsMarshal.AsSpan(artikli);
    if (artikliSpan.Length == 0)
        return 0m;

    var aggregate = IzracunajTezinskiProsjek(artikliSpan, kategorijaId);
    return aggregate.totalQty == 0 ? 0m : aggregate.weightedSum /
       aggregate.totalQty;
}
```

Helper metode:

```
// Extract Method - validacija
private void ValidirajKategoriju(int kategorijaId)

// Extract Method - izracun prosjeka
private static (decimal weightedSum, int totalQty)
    IzracunajTezinskiProsjek(ReadOnlySpan<Artikal?> artikli, int
       kategorijaId)
{
    // Iteracija preko span-a
    for (int i = 0; i < artikli.Length; i++)
    {
        if (!JeValidanArtikal(artikli[i], kategorijaId, out var cijena,
           out var qty))
            continue;
        // ...
    }
}

// Move Complex Boolean Expression
private static bool JeValidanArtikal(Artikal? artikal, int kategorijaId,
    out decimal cijena, out int qty)
{
    if (artikal is null || artikal.KategorijaId != kategorijaId ||
        artikal.Cijena <= 0m || artikal.KolicinaNaStanju <= 0)
    {
        cijena = 0m;
        qty = 0;
        return false;
    }
    cijena = artikal.Cijena;
    qty = artikal.KolicinaNaStanju;
    return true;
}
```

5.4.3 Checklist za Refaktoring

Promjena	Checklist Stavka
ValidirajKategoriju()	Extract a Routine
IzračunajTezinskiProsjek()	Extract a Routine
JeValidanArtikal()	Move Complex Boolean Expression into Well-Named Function
Rani return za praznu kolekciju	Return as Soon as You Know the Answer
Korištenje span iteracije	Data Level - Replace collection traversal with span

Tabela 52: Mapiranje promjena na checklist stavke

5.4.4 Uporedba Metrika

Metrika	Prije (Tuned)	Poslije (Refactored)	Promjena
Ciklomatska kompleksnost (glavna)	19	3	-28.6% do -33.3%
Maintainability Index (glavna)	43	69	+7.3%
Lines of Code (glavna)	78	18	-56.3%

Tabela 53: Uporedba metrika prije i poslije refaktoringa

Poboljšanje:

- Glavna metoda: Kompleksnost smanjena sa 19 na 3
- Maintainability Index: Poboljšanje sa 43 na 69
- Kompleksnost je raspodijeljena na tri male metode sa CC 1-2
- Guard clausule u helperima smanjuju ugniježđenost i daju jasne tačke izlaza

5.4.5 Analiza Utjecaja na Metrike

Extract a Routine:

- Smanjuje kompleksnost glavne metode - validacija i izračun prosjeka su izdvojeni
- Povećava Maintainability Index - kraća glavna metoda ima bolji MI
- Povećava modularnost - svaka metoda ima jasnu, jedinstvenu odgovornost
- Lakše testiranje - svaka metoda može biti testirana nezavisno

Move Complex Boolean Expression:

- Smanjuje ciklomatsku kompleksnost - složeni izraz se broji kao jedna grana
- Poboljšava čitljivost - naziv funkcije jasno opisuje šta se provjerava
- Lakše razumijevanje logike - `JeValidanArtikal` je jasniji od složenog boolean izraza
- Out parametri omogućavaju vraćanje keširanih vrijednosti

Return as Soon as You Know the Answer:

- Smanjenje ugniježđenosti - rani return za praznu kolekciju
- Poboljšanje čitljivosti - jasno se vidi "happy path"
- Smanjenje kompleksnosti - manje grana u kontrolnom toku

Replace Collection Traversal with Span:

- Bez novih alokacija - span je stack-allocated
- Brzi prolaz - direktan pristup elementima bez enumerator overhead-a
- Nema uticaja na CC glavne metode - logika je u helper metodi
- Zadržava performanse tunirane verzije

6 Zaključak

U ovom izvještaju prikazali smo rezultate analize četiri metode iz konzolne aplikacije za upravljanje inventarom. Za svaku metodu smo:

- Izračunali McCabe metrike i konstruisali kontrolne grafove, upoređujući rezultate sa Code Metrics alatom
- Proveli white box testiranje primjenom svih primjenjivih pristupa
- Izvršili code tuning primjenom minimalno 3 različite tehnike, mjereći vrijeme izvršavanja i alociranu memoriju
- Izvršili refaktoring sa ciljem poboljšanja metrika koda

Rezultati pokazuju da primjena različitih tehnika optimizacije i refaktoringa može značajno poboljšati performanse i metrike koda, ali je važno pažljivo testirati svaku promjenu kako bi se osiguralo da ne dolazi do degradacije performansi ili funkcionalnosti.

A Dodatni Materijali

A.1 Rezultati Performance Testova

Detaljni rezultati performance testova za sve verzije metoda mogu se naći u fajlu `PERFORMANCE.md`.

A.2 Izvorni Kod

Kompletan izvorni kod aplikacije i testova dostupan je u repozitoriju projekta.